FC Fluxo de Caixa

DOCUMENTO 04

Decisões arquiteturais (ADRs)

Registro das 31 decisões, com contexto, alternativas e consequências

PROJETO

Fluxo de Caixa — Desafio de Arquiteto de Soluções

AUTOR

Bruno Aquino

DATA

11/11/2026

REPOSITÓRIO

github.com/brunoaquino/fluxo-caixa

Java 25 · Spring Boot 4 · AWS ECS Fargate · Terraform · C4 · DDD · ADRs

Sumário

Decisões de arquitetura 4

Registrar decisões de arquitetura em ADR 6

Microsserviços por bounded context 8

CQRS e integração assíncrona orientada a eventos

Transactional outbox 12

SNS e SQS como mensageria 14

PostgreSQL no RDS, um banco por serviço 16

Java 25 e Spring Boot 4 18

Arquitetura hexagonal 20

ECS Fargate 22

Terraform como IaC 24

OAuth2/OIDC com Cognito e JWT validado no serviço

Cache Redis/ElastiCache no consolidado 28

Observabilidade com OpenTelemetry, ADOT, CloudWatch e X-Ray

Idempotência na escrita e no consumo 32

Lançamentos imutáveis, correção por estorno 34

Representação monetária 36

API REST versionada e problemas RFC 9457 38

Borda com ALB e AWS WAF 40

Região sa-east-1 e LGPD 42

Estratégia de deploy 44

Região us-east-1, com transferência internacional

Borda CloudFront, WAF e VPC Origin 50

Java 25, Spring Boot 4.0.9 e Resilience4j 2.4.0 53

Rotação de segredos 56

Rastreabilidade e propagação de contexto 59

ECS agora, EKS como evolução 61

Observabilidade: gerenciada agora, SaaS ou self-hosted depois

GitOps com ArgoCD, como evolução 66

Três ambientes e state remoto do Terraform 68

Réplica de leitura e estratégia de dados 71

API Gateway REST regional atrás do CloudFront 74

FONTE: DOCUMENTOS-BRUTOS/04-ADR/README.MD

Decisões de arquitetura

Formato MADR. Data de todas: **2026-11-10**. O contrato (nomes, portas, JSON) permanece no blueprint. Estes textos registram a alternativa e o preço.

Decisão substituída não é apagada. O status aponta o ADR vigente.

ADR	Status	Decisão
0001	Aceito	Registrar decisão neste formato, no repositório
0002	Aceito	Dois microsserviços, um por bounded context
0003	Aceito	CQRS e integração por evento, sem chamada síncrona entre os serviços
0004	Aceito	Outbox transacional com relay, em vez de dual-write ou CDC
0005	Aceito	SNS + SQS Standard + DLQ
0006	Aceito	PostgreSQL 17, instância RDS por serviço em produção
0007	Substituído pelo 0023	Java 25 e Spring Boot 4, sem patch pinado
0008	Aceito	Portas e adaptadores, vigiados por ArchUnit
0009	Aceito, ver 0026	ECS Fargate (ARM), mínimo de duas tasks
0010	Aceito, ver 0029	Terraform, provider AWS 6.x
0011	Aceito	OIDC com Cognito e Keycloak; JWT validado no serviço
0012	Aceito	Cache-aside com Redis/Valkey só no consolidado
0013	Aceito, ver 0025 e 0027	OpenTelemetry, ADOT, CloudWatch e X-Ray
0014	Aceito	`Idempotency-Key` no POST e consumidor idempotente por `eventId`
0015	Aceito	Lançamento imutável; correção só por estorno
0016	Aceito	`BigDecimal` / `NUMERIC(19,2)` ; string no evento
0017	Aceito	`/api/v1` e problemas RFC 9457
0018	Substituído pelo 0022	ALB público + WAF regional
0019	Substituído pelo 0021	Região `sa-east-1` por residência do dado
0020	Aceito, ver 0028	Rolling com circuit breaker; blue/green como evolução
0021	Aceito	Região `us-east-1` , transferência com salvaguarda
0022	Aceito, ver 0031	CloudFront, WAF, VPC Origin e ALB interno
0023	Aceito	Boot 4.0.9 e Resilience4j 2.4.0
0024	Aceito	Rotação de RDS, KMS, ACM, JWKS e OIDC

ADR	Status	Decisão
0025	Aceito	W3C e correlation id através da outbox e da fila
0026	Aceito	ECS agora; EKS acima de 8–10 serviços
0027	Aceito	CloudWatch e X-Ray agora; SaaS ou self-hosted depois
0028	Aceito	GitOps com ArgoCD quando houver EKS
0029	Aceito	`dev`, `hml`, `prod`, state em S3
0030	Aceito	Réplica desligada até gatilho; projeção já é o read model
0031	Aceito	REST API regional atrás do CloudFront, VPC Link v2 no ALB

Leitura mínima para o RNF crítico: 0002, 0003, 0004, 0005 e 0014. Para a borda: 0018 (histórico), 0022 e 0031.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0001-REGISTRAR-DECISOES-COM-ADR.MD

Registrar decisões de arquitetura em ADR

- **Status:** Aceito
- **Data:** 2026-11-10

Contexto e problema

O desafio pede justificativa das escolhas, e o sistema vai ser lido por quem não participou da discussão. Sem um lugar estável para o “por quê”, a decisão migra para o comentário do pull request e se perde. O blueprint fixa o contrato; ele não substitui o registro da alternativa rejeitada.

Drivers

- A banca e o time futuro precisam ver opções, não só o resultado.
- A decisão tem de ser curta o bastante para ser lida, e estável o bastante para ser citada.
- O formato precisa caber em Markdown no repositório, sem ferramenta extra.

Opções consideradas

1. Só o README e o blueprint

- Positivo: um arquivo só, sem processo.
- Negativo: o README vira depósito. Trade-off antigo some na próxima edição. Não dá para apontar “a decisão X” numa revisão.

2. Wiki ou página fora do Git

- Positivo: fácil de comentar.
- Negativo: diverge do código. O desafio exige a documentação no repositório. A página externa não viaja com o commit que mudou a fila.

3. ADR no formato MADR, versionado em `documentos-brutos/04-adr`

- Positivo: contexto, drivers, opções com prós e contras, decisão e consequências no mesmo texto. O diff do Git é o histórico. Dá para marcar uma decisão como substituída sem apagar o passado.
- Negativo: mais um documento para atualizar quando a decisão muda. ADR esquecido vira mentira educada.

Decisão

Opção 3. Cada decisão relevante desta entrega tem um ADR numerado, status **Aceito** e data **2026-11-10**. Decisão nova não reescreve a antiga: acrescenta um ADR que a substitui e aponta o número anterior.

O blueprint continua sendo a fonte do contrato (nomes, portas, JSON). O ADR é a fonte do porquê. Se os dois discordarem no contrato, vale o blueprint e o ADR é corrigido.

Consequências

Positivas

- Trade-off fica explícito (SNS contra Kafka, ALB contra API Gateway, região contra custo).
- Dá para revisar uma decisão sem reler a arquitetura inteira.

Negativas

- Trinta arquivos para manter alinhados ao código. Uma decisão implementada diferente do ADR é defeito de documentação, tratado como defeito. Decisão substituída permanece no repositório com status Superseded.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0002-MICROSSERVICOS-POR-BOUNDED-CONTEXT.MD

Microsserviços por bounded context

- **Status:** Aceito
- **Data:** 2026-11-10

Contexto e problema

O enunciado pede dois serviços — lançamentos e consolidado diário — e exige que o primeiro continue disponível se o segundo cair. Há três desenhos capazes de expressar essa separação, com custos operacionais muito diferentes para um time pequeno e um pico de 50 req/s.

Drivers

- Falha do consolidado não pode derrubar o POST de lançamento (processo, banco e deploy).
- O volume não justifica uma plataforma de dezenas de serviços.
- Dois modelos de dados com ciclos de mudança diferentes: trilha imutável e projeção recomputável.
- Operação que caiba em ECS, dois pipelines e um time só.

Opções consideradas

1. Monolito modular

Dois módulos, um processo, um banco ou dois schemas no mesmo Postgres.

- Positivo: uma transação, um deploy, depuração simples, custo de uma instância. A fronteira hexagonal cabe dentro do monolito.
- Negativo: o processo é um só. Um crash, um pool esgotado ou um deploy ruim leva os dois módulos juntos. Isso falha o RNF crítico mesmo com o código “separado”. Isolar de verdade exigiria dois processos — e aí o monolito já deixou de ser a opção.

2. Funções serverless (Lambda por caso de uso)

- Positivo: escala a zero no dev, sem gerir task.
- Negativo: conexão com RDS em rajada (o pico de 50 req/s abre conexões mais rápido do que o pool de um processo quente), cold start na frente do p95 de 100 ms, e VPC attachment para falar com o banco. O relay de outbox pede um processo com relógio, não uma invocação por linha. Virtual threads e o modelo de programação Spring que escolhemos vivem melhor num processo longo. Ver ADR 0007 e ADR 0009.

3. Dois microsserviços, um por bounded context

`lancamentos-service` e `consolidado-service`, cada um com banco, task definition e pipeline.

- Positivo: a queda do consumidor não compartilha processo nem `max_connections` com a escrita. O deploy de um relatório não reinicia o caixa. O mapa de contextos em domínio cabe inteiro nessa fronteira — não há terceiro serviço "porque microsserviço".
- Negativo: dois deploys, dois RDS em produção, consistência eventual, rastreio distribuído obrigatório. Latência do saldo deixa de ser a latência do POST.

Decisão

Opção 3. A fronteira para no par pedido pelo negócio. Não se extrai "estorno", "categoria" ou "comerciante" para outro serviço. Identidade fica no Cognito/Keycloak, que já é um serviço genérico.

Consequências

Positivas

- O teste de resiliência consegue derrubar só o consolidado e ainda receber 201.
- O schema da projeção muda sem migrar a tabela de auditoria.

Negativas

- Custo fixo de duas instâncias RDS e de dois serviços no ALB. Aceito e quantificado em custos.
- Todo bug de saldo agora tem dois logs. Sem o correlation id do ADR 0013, a opção 3 fica pior que o monolito para operar.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0003-CQRS-E-INTEGRACAO-ASSINCRONA.MD

CQRS e integração assíncrona orientada a eventos

- **Status:** Aceito
- **Data:** 2026-11-10

Contexto e problema

O comerciante escreve fatos (lançamento) e lê um agregado derivado (saldo do dia e acumulado). Dá para servir as duas coisas da mesma tabela, com o serviço de leitura chamando o de escrita, ou separar o modelo de leitura e empurrar o fato por evento.

Drivers

- O POST não pode depender do consolidado estar de pé.
- O pico de 50 req/s é de leitura do saldo, não de varredura do extrato.
- Estorno retroativo altera o dia de competência, não “cria um ajuste hoje”.
- Atraso aceitável: p95 < 5 s (blueprint, meta de consolidação).

Opções consideradas

1. Chamada síncrona REST de lançamentos para o consolidado, no meio do POST

- Positivo: o 201 só sai quando o saldo já bate. Um modelo mental.
- Negativo: o RNF crítico fica impossível. Timeout, circuit breaker ou retry no meio do POST ou atrasam o caixa ou mentem sobre o saldo. Acoplamento de disponibilidade e de deploy.

2. Banco compartilhado, consolidado lendo a tabela de lançamentos

- Positivo: sem evento, sem lag, `SUM` sempre fresco.
- Negativo: o consolidado depende do banco de lançamentos. Carga de leitura de pico e um relatório pesado competem com o INSERT do caixa. Não é database-per-service. Um lock ou um disco cheio no relatório afeta a escrita.

3. CQRS com evento `LancamentoRegistrado`

A escrita grava o fato e publica o evento. A leitura mantém `saldos_diarios` e responde o GET sem consultar o outro serviço. A soma é comutativa; ordem não é requisito.

- Positivo: disponibilidade desacoplada, esquema de leitura feito para o GET, espaço para cache, novos consumidores sem mudar o POST.

- Negativo: janela em que o GET ainda não vê o lançamento. O cliente precisa entender consistência eventual. Operação ganha fila, DLQ e lag.

Decisão

Opção 3. O contrato publicado é o JSON versão 1 do blueprint, no tópico `fluxo-caixa-lancamentos-events`. Não há API interna síncrona entre os dois serviços.

O GET de lançamento (extrato) continua no serviço de escrita: é leitura do próprio agregado, não do read model de saldo. CQRS aqui é entre contextos, não uma proibição de ler o que se acabou de escrever no mesmo serviço.

Consequências

Positivas

- O dimensionamento do consolidado (cache, pool, tasks) não entra no SLO do POST.
- Estorno não precisa de endpoint no consolidado: o mesmo evento, com tipo oposto, corrige o dia.

Negativas

- Tela que gravar e reler o saldo no mesmo clique pode ver o valor antigo por até alguns segundos. O contrato da API deixa isso explícito; não se bloqueia o POST até a projeção.
- Teste de ponta a ponta precisa esperar a consolidação ou consultar a projeção com retry. O smoke faz isso com prazo, não com sleep cego infinito.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0004-TRANSACTIONAL-OUTBOX.MD

Transactional outbox

- **Status:** Aceito
- **Data:** 2026-11-10

Contexto e problema

O lançamento e o evento de integração precisam concordar. Se o banco confirma e o SNS não recebe, o saldo nunca vê o fato. Se o SNS recebe e o banco desfaz, o saldo soma um fato que não existe. Os dois sistemas não participam da mesma transação.

Drivers

- Zero perda de evento depois de um 201.
- POST com p95 < 200 ms, portanto o publish não pode ser obrigatório dentro do tempo de resposta.
- Duas réplicas do serviço publicando ao mesmo tempo, sem evento duplicado por corrida do relay.
- Volume da premissa: 20 escritas/s no pico, muito abaixo do que justificaria uma plataforma de CDC.

Opções consideradas

1. Dual-write

`COMMIT` e, em seguida, `Publish`.

- Positivo: pouco código.
- Negativo: o processo pode morrer no meio. Retry do publish depois do commit exige exatamente a caixa de saída que esta opção finge não precisar. Publicar antes do commit entrega evento de transação abortada.

2. CDC com Debezium lendo o WAL do Postgres, publicando no SNS ou no Kafka

- Positivo: o domínio não conhece outbox. O log é a fonte. Bom quando há muitos agregados e vários consumidores de banco legado.
- Negativo: um conector para operar, em geral em Kafka Connect ou MSK Connect, com o custo e a complexidade que o ADR 0005 já recusou para a mensageria. Para uma tabela e um evento, é mecanismo demais. Também empurra o formato do evento para o formato da linha, a menos que um SMT o traduza — a tradução volta a ser código nosso.

3. Outbox na mesma transação e relay no processo

Tabela `outbox_eventos(id, aggregate_id, event_type, payload JSONB, criado_em, publicado_em, tentativas, proxima_tentativa_em, trace_parent, trace_state, correlation_id)`. O caso de uso insere o lançamento e a linha, com `proxima_tentativa_em` igual ao instante da escrita, e grava o contexto de

trace da requisição (ADR 0025). Um relay agenda a cada 500 ms, lê lote de 100 com `SELECT … FOR UPDATE SKIP LOCKED` das linhas ainda não publicadas cuja próxima tentativa já chegou, publica e marca `publicado_em`. Falha de publish incrementa `tentativas` e empurra `proxima_tentativa_em` com backoff exponencial até 60 s. O cliente SNS do relay ainda tem CircuitBreaker, Retry com jitter e TimeLimiter (ADR 0023). O POST continua sem chamar o SNS.

- Positivo: o 201 só sai se o evento está durável. Falha do SNS atrasa a publicação e não desfaz o lançamento. `SKIP LOCKED` permite duas tasks. O payload é o JSON do contrato, não um dump da tabela.
- Negativo: polling (atraso médio de uma fração do intervalo de 500 ms, compatível com o p95 de 5 s de ponta a ponta). Tabela que cresce se o publish parar — precisa de alarme de idade e de limpeza das linhas já publicadas. Relógio dentro da API, que compete um pouco de CPU com o POST (irrelevante a 20 escritas/s).

Decisão

Opção 3. O relay vive no `lancamentos-service`, não num terceiro deploy. Extrair um worker só se a métrica mostrar que o agendador atrapalha a API.

Publicação bem-sucedida atualiza `publicado_em` numa transação curta depois do ack do SNS. Se o processo cair depois do publish e antes do update, o evento é republicado. O consumidor idempotente (ADR 0014) torna essa duplicata inofensiva.

Consequências

Positivas

- O RNF de durabilidade do lançamento e o de independência do consolidado ficam no mesmo mecanismo.
- O teste de integração sobe Postgres e LocalStack e prova o ciclo sem AWS de verdade.

Negativas

- Relógio de 500 ms e lote de 100 são parâmetros operacionais. Mudá-los mexe no lag e na carga do banco. Ficam em configuração, com o alarme de outbox pendente do documento 07.
- A limpeza de linhas publicadas é tarefa de manutenção. Sem ela, a tabela cresce para sempre. Retenção sugerida: 7 dias depois de `publicado_em`, para auditoria curta do payload.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0005-SNS-SQS-MENSAGERIA.MD

SNS e SQS como mensageria

- **Status:** Aceito
- **Data:** 2026-11-10

Contexto e problema

O outbox precisa de um transporte entre o serviço de lançamentos e o de consolidado, com retenção se o consumidor cair, redrive para mensagem inválida e espaço para um segundo consumidor no futuro. A premissa de volume é 20 eventos/s no pico de escrita e cerca de 2 milhões de eventos por mês se a média for ~0,8 evt/s. O requisito de leitura (50 req/s) não passa pela fila.

Drivers

- Retenção de muitos dias (o blueprint fixa 14 na fila).
- DLQ com `maxReceiveCount` 5 e visibility timeout de 60 s.
- Fan-out: um tópico, uma fila hoje, outra fila amanhã sem o produtor mudar.
- Custo e operação compatíveis com dois serviços, não com um cluster.
- LocalStack no ambiente local, SDK v2 na aplicação.
- Ordem total não é necessária: a soma é comutativa (domínio).

Opções consideradas

1. Amazon MSK (Kafka)

- Positivo: log retido, replay por offset, vários consumidores independentes, ecossistema de stream.
- Negativo: o menor cluster Multi-AZ são três brokers mais armazenamento, na casa de **centenas de USD por mês** mesmo em `us-east-1` (e mais caro em `sa-east-1`), duas ordens de grandeza acima da conta de SNS/SQS neste volume. Operação de versão, disco e partição. Local fica mais pesado que um LocalStack. Kafka se paga quando houver replay longo, processamento de stream ou throughput sustentado na casa de milhares de eventos/s — cenário do roadmap, não o de agora.

2. Amazon MQ (ActiveMQ ou RabbitMQ)

- Positivo: fila clássica, roteamento flexível, protocolo que muitos ERPs já falam.
- Negativo: broker para patchar, dimensionar e colocar em Multi-AZ (ativo/espera custa instância dobrada). Não é o encaixe natural do restante da conta, que já é API AWS. Mais caro e mais frágil que SQS para um único consumidor idempotente.

3. Amazon EventBridge

- Positivo: roteamento por regra, schema registry, baixo custo na casa de USD 1 por milhão de eventos.

- Negativo: o buffer de consumidor, a concorrência e a DLQ que queremos são do SQS. EventBridge na frente do SQS acrescenta um hop e uma conta sem uma regra de roteamento que o SNS já não faça com assinatura. Útil no dia em que dezenas de tipos de evento precisarem de filtros diferentes. Hoje existe um tipo.

4. SNS + SQS Standard + DLQ

Tópico `fluxo-caixa-lancamentos-events`, fila `consolidado-lancamentos-queue`, DLQ `consolidado-lancamentos-dlq`. Assinatura com raw message delivery. Fila Standard, não FIFO.

- Positivo: serviço gerenciado, retenção de 14 dias, long polling, competing consumers nas tasks do consolidado, custo de poucos dólares no volume da premissa (a linha está em custos, abaixo de USD 10/mês). Segundo consumidor = segunda fila na mesma assinatura. LocalStack cobre o fluxo.
- Negativo: sem ordem e sem replay por offset. Mensagem consumida e apagada não volta, salvo retenção ainda não expirada ou redrive da DLQ. Payload máximo 256 KB — o evento v1 está muito abaixo. Duplicata é normal e tem de ser absorvida pelo consumidor.

FIFO foi descartado dentro desta opção. FIFO dá ordem por message group e deduplicação de 5 minutos, com teto de throughput por grupo. A deduplicação curta não substitui a tabela `eventos_processados` (o redrive acontece dias depois). A ordem não é requisito. Standard é o encaixe.

Decisão

Opção 4, fila Standard, criptografia KMS, policy mínima por task role. Cliente AWS SDK for Java v2, sem Spring Cloud AWS, para não acoplar o ciclo de vida do framework ao binding da fila. O listener é código nosso no adaptador, pequeno o bastante para testar com LocalStack.

Consequências

Positivas

- O custo variável de mensageria é irrelevante perto de NAT, endpoint e RDS. A decisão não precisa ser revista por dinheiro até o volume subir ordens de grandeza.
- O teste de resiliência enche a fila com o consumidor parado e depois observa o catch-up.

Negativas

- Sem log de eventos consultável por offset. Reprocessar história inteira exige reler a outbox ou o banco de lançamentos e republicar — procedimento de operação, não feature do ônibus. Se um dia o reprocessamento histórico for rotina, a opção 1 volta à mesa.
- Visibilidade e alarme passam a ser parte do desenho. Fila sem alarme de idade é evento perdido na prática, mesmo com retenção de 14 dias.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0006-POSTGRESQL-DATABASE-PER-SERVICE.MD

PostgreSQL no RDS, um banco por serviço

- **Status:** Aceito
- **Data:** 2026-11-10

Contexto e problema

Os dois serviços persistem estado relacional com integridade (lançamento imutável, unicidade de estorno, saldo por UPSERT atômico). É preciso escolher o motor e o grau de isolamento: schema, database ou instância.

Drivers

- Transação local forte na escrita e na aplicação do evento.
- `NUMERIC` para dinheiro, índice único parcial para estorno, `SKIP LOCKED` para o relay, JSONB para o payload da outbox.
- Falha de capacidade do consolidado (conexão, disco, IOPS) não pode impedir o INSERT de lançamento.
- RPO ≤ 5 min e recuperação pontual.
- Time que já opera SQL. Volume que cabe numa instância small (ver capacidade).

Opções consideradas

1. Um RDS, dois databases

- Positivo: uma instância, uma janela de backup, metade do custo de compute Multi-AZ.
- Negativo: CPU, disco e `max_connections` são compartilhados. O isolamento que o RNF pede no processo se desfaz no banco. Serve para o Docker local e, se o orçamento apertar, para um dev AWS. Não serve para produção.

2. DynamoDB por serviço

- Positivo: escala horizontal gerenciada, sem conexão persistente.
- Negativo: o relay com `SKIP LOCKED`, a unicidade condicional do estorno e o `SUM` do acumulado viram desenho de chave e GSI. Transação de até 100 itens não é o problema; a perda é o modelo relacional que expressa as invariantes com pouco código. O time pagaria complexidade de modelagem num volume que não precisa dela.

3. PostgreSQL 17, instância RDS por serviço em produção

Dois bancos: `lancamentos` e `consolidado`. Em produção, duas instâncias Multi-AZ, criptografadas com KMS, senha no Secrets Manager, Flyway em cada serviço. Classe inicial `db.t4g.small` (2 GiB, ~225 conexões), porque o cálculo de Little dá menos de uma conexão ativa no pico e no máximo 60 reservadas no teto

de tasks. gp3 50 GB.

- Positivo: SQL, integridade, PITR, failover Multi-AZ, motor maduro para `NUMERIC(19,2)`. Instância separada realiza o database-per-service de verdade. Small é suficiente e custa cerca de metade de medium nesta região (números no documento 08).
- Negativo: duas contas de RDS, o maior item variável junto com a rede. Multi-AZ dobra o preço da hora da instância e o GB do gp3 já vem precificado em dobro na tabela (`SAE1-RDS:Multi-AZ-GP3-Storage`). Operação de versão do Postgres em dois lugares.

Decisão

Opção 3 em produção. No Docker local, um único Postgres cria os dois databases e os usuários `lancamentos` e `consolidado` — o isolamento de credencial existe, o de hardware não, e isso é aceitável na máquina do desenvolvedor. Dev AWS, se existir, usa duas instâncias `db.t4g.micro` Single-AZ para manter a fronteira de segredo sem pagar Multi-AZ.

Não há usuário de banco cruzado. Não há foreign key entre serviços. Não há dblink.

Consequências

Positivas

- A invariante “um estorno” é índice único, não só `if` no código.
- Dá para restaurar o consolidado de um PITR sem rebobinar a trilha de lançamentos, e o contrário.

Negativas

- Relatório que precise de extrato e saldo no mesmo request faz duas chamadas HTTP. Não faz join. É o custo do isolamento e é o que a API já expressa.
- Subir de classe (small → medium) é a resposta a CPU credit ou memória, não a um pressentimento. O gatilho é métrica, documentado no runbook.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0007-JAVA-25-SPRING-BOOT-4.MD

Java 25 e Spring Boot 4

- **Status:** Substituído (Superseded) pelo ADR 0023
- **Data:** 2026-11-10

O texto abaixo escolheu Java 25 e a linha Boot 4 sem pinar o patch e sem Resilience4j. A decisão vigente é Boot 4.0.9 e Resilience4j 2.4.0. A frase deste arquivo que diz que a versão de patch não está pinada não vale mais.

Contexto e problema

O serviço é I/O-bound: JDBC, Redis, JWKS e SNS no meio de poucas dezenas de requisições simultâneas (Little dá 2,8 em voo no pico de leitura). A linguagem precisa expressar o domínio sem ruído, rodar em container enxuto e ter suporte longo.

Drivers

- LTS. Java 25, publicado em setembro de 2025, é a LTS seguinte à 21.
- Virtual threads para o bloqueio de JDBC e HTTP sem um pool de plataforma dimensionado no chute.
- Ecossistema que o time opera: Spring, Flyway, Testcontainers, ArchUnit.
- Imagem de container reproduzível. JDK de distribuição com licença que não cobra por núcleo.

Opções consideradas

1. Java 21 LTS e Spring Boot 3.x

- Positivo: curva conhecida, mais tempo de mercado, menos surpresa de biblioteca.
- Negativo: abre mão da LTS atual e do alinhamento com Spring Boot 4, que é a linha definida no blueprint. Manter 21 “por segurança” em projeto novo em 2026 é dívida de largada.

2. Kotlin ou outra JVM language

- Positivo: null-safety na linguagem.
- Negativo: segundo dialeto para um domínio que cabe em Java claro. Não há requisito que Kotlin resolva e Java 25 não resolva.

3. Java 25 e Spring Boot 4.x na última estável do Maven Central

Runtime Eclipse Temurin 25 (licença de uso sem custo por núcleo; ver custos). Um Maven wrapper por serviço. Virtual threads ligadas no servidor web e nos clientes bloqueantes.

- Positivo: LTS, virtual threads maduras, um processo aguenta o pico de 50 req/s folgado porque o limite deixa de ser thread de plataforma. Spring Boot 4 é a linha que acompanha essa geração de Jakarta e do

ecossistema.

- Negativo: biblioteca que ainda não subiu para a linha 4 quebra o build. A versão de patch não está pinada no blueprint de propósito ("última estável"); o código deve pinar a que resolver o build e registrar no `pom`. Este ADR não inventa número de patch.

Decisão

Opção 3. Dois projetos Maven independentes, pacotes `br.com.fluxocaixa.lancamentos` e `br.com.fluxocaixa.consolidado`. Imagem com Temurin 25, usuário não-root, JAR em camadas.

Virtual threads não autorizam pool JDBC infinito. O Hikari continua limitado (máximo 10 por task), como calculado na capacidade. Thread barata com conexão cara só muda o lugar do acidente.

Consequências

Positivas

- O modelo de concorrência combina com o perfil medido: poucas requisições em voo, cada uma esperando rede.
- Um wrapper Maven tira a exigência de Maven instalado na máquina, desde que o JDK 25 exista.

Negativas

- Se uma dependência crítica não publicar artefato compatível com Spring Boot 4, a mitigação é subir o patch ou, em último caso, registrar um ADR novo — não misturar Boot 3 num serviço e Boot 4 no outro.
- JDK comercial da Oracle fica fora da imagem. Entrar com ele criaria custo de licença que a estimativa trata como zero.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0008-ARQUITETURA-HEXAGONAL.MD

Arquitetura hexagonal

- **Status:** Aceito
- **Data:** 2026-11-10

Contexto e problema

O domínio de lançamento (dinheiro, estorno, imutabilidade) é a parte que a banca e o negócio vão questionar. Esse código não pode depender de anotação de framework, de cliente AWS nem de tabela. Ao mesmo tempo, a aplicação precisa de HTTP, JPA, SNS e Redis de verdade.

Drivers

- Teste de invariante sem subir Spring.
- Troca de LocalStack por AWS sem editar regra de estorno.
- Regra de dependência que o build verifica, não uma convenção de pasta que alguém fura na pressa.
- Pacotes já nomeados no blueprint.

Opções consideradas

1. Camadas técnicas clássicas (`controller` , `service` , `repository`) com domínio anêmico

- Positivo: familiar, menos arquivos.
- Negativo: a invariante de estorno acaba no service que já importa o SDK, e o teste vira `@SpringBootTest` para qualquer regra. Em seis meses o “service” conhece fila, token e SQL.

2. Pacotes por feature, sem restrição de compilação

- Positivo: navegação fácil.
- Negativo: a fronteira é social. Não impede `domain` de importar `software.amazon.awssdk` .

3. Portas e adaptadores, com ArchUnit

```
domain/          entidades, value objects, eventos, exceções — sem Spring
application/     casos de uso e portas de saída
adapters/in      web e, no consolidado, messaging
adapters/out     persistence, messaging, cache
config/
```

ArchUnit falha o build se `domain` depender de Spring ou de `adapters` , e se `application` depender de `adapters` .

- Positivo: a regra de estorno mora em `Lancamento` e roda em teste de unidade. O JSON do evento mora no adaptador de mensagem (ACL). O mesmo caso de uso serve ao teste de contrato e ao controller.
- Negativo: mais interfaces e mapeamentos DTO ↔ domínio ↔ entidade JPA. Para dois agregados pequenos, parte dessa estrutura parece cerimônia. É o preço de manter a regra legível quando o adaptador AWS crescer.

Decisão

Opção 3, exatamente nos pacotes do blueprint. O controller não é lugar de regra: extrai o `comerciante_id` do JWT, monta o comando e traduz falha de domínio em problema RFC 9457.

Não se cria porta de saída “por precaução” para um barramento genérico. As portas que existem são as que têm adaptador: repositório, outbox, publish, cache, fila.

Consequências

Positivas

- O nível 4 do C4 é legível porque o domínio não está emaranhado no filtro de segurança.
- Teste de arquitetura pega o atalho antes do code review.

Negativas

- ArchUnit mal escrito vira ruído (falso positivo em `config`). O teste cobre os três pacotes de regra e deixa `config` como composição, que por natureza conhece todo mundo.
- Mapear `Dinheiro` ↔ `NUMERIC` ↔ string do evento em três bordas é repetição consciente. Compartilhar um módulo de domínio entre os serviços recriaria um kernel compartilhado que o context map recusou.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0009-ECS-FARGATE.MD

ECS Fargate

- **Status:** Aceito
- **Data:** 2026-11-10

Contexto e problema

Os dois processos Java precisam rodar em `us-east-1` (ADR 0021), em pelo menos duas zonas, com deploy repetível, escala horizontal e sem uma frota de máquina para patchar. O pico de CPU é baixo (ρ ≈ 17% com duas tasks no consolidado). O ADR 0026 mantém esta decisão e registra o EKS como evolução.

Drivers

- Mínimo de duas tasks em AZs distintas por serviço.
- Imagem imutável vinda do ECR, scan no push.
- Rolling com circuit breaker e rollback.
- Sidecar ADOT no mesmo task.
- Custo proporcional a task pequena, arquitetura ARM.
- Tempo de plataforma que sobre para o domínio, não para o control plane.

Opções consideradas

1. EKS

- Positivo: padrão de mercado, HPA, ecossistema, portabilidade de manifesto.
- Negativo: control plane pago o mês inteiro (na casa de USD 70 só de API do cluster, mais nós ou Fargate profile). Para quatro tasks, o Kubernetes é o sistema maior da conta. A portabilidade não é requisito: o alvo declarado é AWS.

2. Lambda

- Positivo: sem task ociosa.
- Negativo: já recusado no ADR 0002 por conexão com RDS, cold start e relay de outbox. Não se reabre aqui.

3. EC2 com AMI e autoscaling group

- Positivo: preço de compute menor que Fargate no uso constante, controle do SO.
- Negativo: patch de AMI, capacidade ociosa do nó, user data, e um ASG que ainda precisa de load balancer. Com ρ de 17%, o desconto do EC2 não paga a operação. Reservar instância para dois serviços pequenos é otimização prematura; Savings Plan de Fargate, se o uso estabilizar, vem antes.

4. ECS Fargate, graviton (ARM)

Cluster único, um service por microsserviço. Em produção o lançamentos sobe com 0,5 vCPU e 1 GiB (mínimo 2, máximo 6) e o consolidado com 1 vCPU e 2 GiB (mínimo 2, máximo 10). Aplicação e sidecar ADOT dividem esse tamanho. Escala por target tracking de CPU (60%) e de `ALBRequestCountPerTarget` nos dois. O consolidado soma target tracking do backlog da fila (`mensagens visíveis / tasks em execução` , alvo de 20). Deployment circuit breaker com rollback. Container Insights. Dev usa tasks menores e uma task por serviço. Os números estão em `infra/terraform/envs/*/terraform.tfvars.example` .

- Positivo: sem nó para administrar, task spread nas AZs, encaixe direto com ALB target group, IAM por task role, imagem OCI padrão. ARM no Fargate de `us-east-1` custa USD 0,03238 por vCPU-hora e USD 0,00356 por GB-hora (Price List, vigência 2026-10-01), abaixo da hora x86 da mesma família. Em `sa-east-1` os mesmos SKUs são USD 0,05570 e USD 0,00612.
- Negativo: menos controle de kernel e de daemon. Sidecar come parte da CPU contratada — por isso o consolidado, que segura o pico de 50 req/s, fica em 1 vCPU e não em 0,25. O lançamentos, com 20 escritas/s, cabe em 0,5 vCPU. Cold start de task nova é da ordem de dezenas de segundos; o mínimo de 2 tasks em prod existe para não depender de scale-from-zero no pico. Fargate não fica de graça parado. O exemplo de dev não desliga a task sozinho.

Decisão

Opção 4. Um cluster, dois services, dois target groups. Não há service mesh. A task não recebe IP público; egress por NAT ou por VPC endpoint, como no deployment.

Consequências

Positivas

- O N−1 do RNF (uma task morta e o pico de 50 req/s segue) é configuração de desired count e de spread, não um script.
- A mesma imagem sobe no Docker local e no ECR.

Negativas

- Debug de produção é log, trace e task replacement, não SSH. Quem precisar de shell usa ECS Exec pontual, com IAM, não uma chave na AMI.
- Teto de 6 tasks no lançamentos e de 10 no consolidado é guarda-corpo de custo. Subir o teto sem olhar RDS e Redis só multiplica pool de conexão.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0010-TERRAFORM-IAC.MD

Terraform como IaC

- **Status:** Aceito. O ADR 0029 amplia os ambientes para `dev`, `hml` e `prod` e fixa o state em S3. A escolha do Terraform permanece.
- **Data:** 2026-11-10

Contexto e problema

VPC, ALB, WAF, ECS, dois RDS, ElastiCache, SNS, SQS, Cognito, alarmes e IAM precisam ser recriáveis em `dev` e `prod`, revisáveis em pull request e destruíveis. A região e os nomes de fila já estão fixados no blueprint.

Drivers

- Estado declarativo e plan visível antes do apply.
- Módulos reutilizáveis entre ambientes, com tamanho diferente (NAT único no dev, NAT por AZ na prod).
- Ferramenta que o repositório expressa em HCL, sem gerar JSON ilegível à mão.
- Sem custo de licença para o porte deste sistema. A licença da ferramenta em si está tratada no documento 08: Terraform é BSL, uso gratuito neste contexto; o runtime não embute a CLI.

Opções consideradas

1. AWS CDK

- Positivo: tipos, loops e testes na linguagem do time (TypeScript ou Java), construct de alto nível.
- Negativo: a unidade de review deixa de ser a infraestrutura e passa a ser o programa que a gera. Diff de CloudFormation sintetizado é mais difícil de ler do que diff de HCL. Mais uma cadeia de build para a infra.

2. CloudFormation puro

- Positivo: nativo, sem estado externo além do stack, drift detection da AWS.
- Negativo: verbosidade, módulos fracos comparados ao que os dois ambientes precisam, e templates grandes de VPC/ECS que a comunidade já resolveu em módulo Terraform. Rollbacks de stack têm arestas conhecidas com recurso que não volta atrás (RDS).

3. Terraform (provider AWS 6.x), módulos e ambientes `dev` e `prod`

Estado remoto (S3 + lock), `make tf-validate` no CI, um módulo por preocupação (rede, borda, serviço ECS, dados, mensageria, identidade, observabilidade). Nomes de fila e tópico iguais aos do blueprint, para a aplicação e a infra não divergirem.

- Positivo: plan legível, ecossistema de módulos, o mesmo fluxo nos dois ambientes, provider 6.x alinhado ao blueprint. Validate e fmt cabem no pipeline sem aplicar nada.

- Negativo: estado é um artefato crítico (criptografado, lock, sem commit do `tfstate`). Provider 6.x muda argumento em relação à linha 5; o código não copia exemplo antigo de blog. HCL não impede um `apply` de prod a partir da máquina errada — isso é política de CI, não de sintaxe.

Decisão

Opção 3. Apply de produção só pelo pipeline, com o ambiente `prod` protegido. Apply manual fica restrito a `dev`. O README de `infra/terraform` é o passo a passo; este ADR só registra a escolha da ferramenta.

OpenTofu permanece como saída se a licença BSL se tornar impeditiva para a organização. O HCL escrito para este porte é portável o bastante para essa troca não ser uma reescrita.

Consequências

Positivas

- A revisão da infra é um diff, não um clique no console.
- Desviar o console sem refletir no código aparece como drift a corrigir, não como “ajuste que ficou”.

Negativas

- Dois lugares definem nome de fila: Terraform e configuração da aplicação. O blueprint é a lista canônica; os dois lados copiam dela. Um teste de fumaça na AWS de dev que publica e consome pega divergência que o `fmt` não pega.
- Recurso com dado (RDS) não entra em destroy casual. O ambiente `prod` não tem destroy no alvo do dia a dia do Makefile.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0011-OAUTH2-OIDC-COGNITO.MD

OAuth2/OIDC com Cognito e JWT validado no serviço

- **Status:** Aceito
- **Data:** 2026-11-10

Contexto e problema

Cada requisição pertence a um comerciante e a um escopo. O identificador do comerciante não pode vir do corpo nem da URL. O ambiente local precisa emitir token sem conta AWS. O ambiente AWS precisa de um provedor gerenciado.

Drivers

- Access token JWT, validado por assinatura e issuer.
- Escopos `lancamentos:write`, `lancamentos:read`, `consolidado:read`.
- Claim `comerciante_id`. Na AWS o prefixo de escopo é `fluxo-caixa/`; no local é vazio. A aplicação lê `app.security.scope-prefix` e `app.security.comerciante-claim`.
- Mesmo código de resource server nos dois ambientes.
- Sem sessão de servidor. A API é stateless.

Opções consideradas

1. API key estática por comerciante

- Positivo: trivial de testar.
- Negativo: sem expiração decente, sem escopo, sem rotação fácil, e o tenant volta a ser um header que o cliente escolhe. Não distingue leitura de escrita.

2. Authorizer só na borda (API Gateway ou ALB authenticate-oidc), serviço confia na rede

- Positivo: o serviço não fala com JWKS.
- Negativo: qualquer caminho que alcance a task (SG folgado, outro serviço no cluster) é chamada anônima com poder de escrita. O comportamento local (sem ALB) divergiria do comportamento AWS. A borda escolhida no ADR 0018 não substitui autorização de aplicação.

3. Cognito na AWS, Keycloak no Docker, resource server em cada serviço

User Pool com resource server `fluxo-caixa` e os três escopos. Pre Token Generation Lambda v2 coloca `comerciante_id` no access token. No local, realm `fluxo-caixa`, client público `fluxo-caixa-app` com direct access grants, usuário `comerciante1` / `comerciante1` e o mesmo claim.

O serviço valida o JWT (Spring Security Resource Server). Dentro do Compose, `JWT_ISSUER_URI` permanece `http://localhost:8080/realms/fluxo-caixa` para coincidir com o `iss` do token obtido no host, e `JWT_JWK_SET_URI` aponta para `http://keycloak:8080/realms/fluxo-caixa/protocol/openid-connect/certs`, porque de dentro da rede o `localhost` do issuer não é o Keycloak.

- Positivo: padrão OIDC, escopo de verdade, tenant fora do alcance do cliente, paridade de código entre local e AWS, Keycloak sem custo de licença para o laboratório.
- Negativo: a Lambda de pre-token é um pedaço de infra a testar — se ela falhar, o token sai sem claim e o serviço responde 401, o que é o comportamento seguro e também um incidente de login. Cognito tem modelo de preço por MAU (dentro da franquia neste porte; ver custos). O arranjo de issuer versus JWK no Compose é fácil de "corrigir" errado: unificar os dois URLs quebra ou a validação do `iss` ou a busca da chave.

Decisão

Opção 3. Sem claim, 401. Escopo ausente, 403. `comerciante_id` de outro formato que não UUID, 401. Não existe comerciante default.

Autorização de objeto é o filtro por esse claim em toda query. Não se cria papel de administrador nesta versão.

Consequências

Positivas

- O teste local (`make token`) exercita o mesmo filtro que a produção.
- Vazar um token não entrega outro comerciante. O estrago fica no tenant e na validade do token.

Negativas

- Direct access grant no client local é senha no POST do token. Aceitável no realm de laboratório, inaceitável no client de produção. Produção usa o fluxo autorizado do canal (authorization code com PKCE quando houver usuário interativo). O client público com senha não é copiado para o Cognito.
- Relógio das tasks precisa estar são. Skew grande demais rejeita token bom. NTP da plataforma Fargate cuida disso; o sintoma, se aparecer, é 401 com token recém-emitido.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0012-CACHE-REDIS-ELASTICACHE.MD

Cache Redis/ElastiCache no consolidado

- **Status:** Aceito. O fallback quando o Redis falha ou abre o circuito está no ADR 0023.
- **Data:** 2026-11-10

Contexto e problema

O SLO de leitura é p95 < 100 ms no GET do saldo, com 50 req/s. O Postgres aguenta a taxa mesmo sem cache (a conta de conexões ativas no miss total é ~2), mas não garante o percentil quando o `SUM` do acumulado e a validação do token disputam a mesma latência. O serviço de lançamentos não tem este SLO de leitura derivada e não ganha cache.

Drivers

- Acelerar o hit sem tornar o Redis fonte da verdade.
- Invalidar dia, acumulado e relatório de período quando um evento de qualquer data daquele comerciante chega (estorno retroativo muda o acumulado dos dias seguintes).
- Perder o cache não pode perder saldo nem derrubar a leitura.
- Produção com failover. Local com um container Redis 7.

Opções consideradas

1. Sem cache

- Positivo: um componente a menos, leitura sempre tão fresca quanto a projeção.
- Negativo: o p95 de 100 ms fica inteiro nas costas do RDS e do cálculo do acumulado. Possível neste volume, folgado demais para tratar como arquitetura — o primeiro relatório de período de 366 dias no meio do fechamento come o percentil.

2. Cache embutido na task (Caffeine)

- Positivo: sem rede extra, latência mínima.
- Negativo: duas tasks, duas memórias. Invalidar a geração no evento só limpa a task que consumiu a mensagem. A outra serve saldo velho até o TTL. Para um read model que acabou de mudar, isso é o bug que o comerciante vê.

3. Redis no local e Valkey no ElastiCache, cache-aside com geração

Chaves `consolidado:versao:{comercianteId}`, `consolidado:dia:{comercianteId}:{versao}:{data}` e `consolidado:relatorio:{comercianteId}:{versao}:{inicio}:{fim}`. Cada evento aplicado faz `INCR` na versão daquele comerciante depois do commit. TTL de 30 s nas entradas (`app.cache.ttl`). ElastiCache Val-

key 8.2, nó `cache.t4g.small` com réplica e failover, TLS (`REDIS_SSL=true`) e sem senha. No Docker local o cache é Redis 7 sem TLS. A aplicação fala protocolo Redis; Valkey é a escolha de produção por compatibilidade e por preço de tabela inferior ao Redis equivalente na AWS.

- Positivo: as duas tasks veem a mesma geração. Estorno antigo não exige apagar um conjunto desconhecido de chaves. Queda do cache: o caso de uso lê o Postgres. Queda do primário: a réplica assume.
- Negativo: mais um serviço gerenciado (cerca de USD 37/mês os dois nós small em `us-east-1` , custos; em `sa-east-1` a mesma dupla ficaria perto de USD 71). Invalidação por versão deixa lixo até o TTL (aceito). Se o `INCR` falhar depois do commit do saldo, a versão antiga continua válida até o TTL — janela de leitura obsoleta de no máximo 30 s. Não se finge que o cache é linearizável com o banco.

Decisão

Opção 3, somente no `consolidado-service` . Variáveis `REDIS_HOST` , `REDIS_PORT` , `REDIS_SSL` . Timeout curto no cliente e fallback para o banco quando o Redis não responde. Não se cacheia lançamento: o GET por id já é uma chave primária e não é o pico dos 50 req/s.

Consequências

Positivas

- O SLO de 100 ms tem um caminho de hit que não abre transação.
- O teste de carga consegue um cenário com cache quente e outro com Redis parado, e os dois têm critério de sucesso diferente (latência versus só disponibilidade).

Negativas

- Mais um security group. Não há `REDIS_PASSWORD` : o Terraform liga TLS e só o SG do consolidado alcança a porta 6379.
- Resposta cacheada de dia zerado existe. Um bug na versão serviria zero por até 30 s depois do primeiro lançamento do dia. O `INCR` depois do commit do evento é o que impede isso; o teste de integração cobre “evento aplicado, GET seguinte vê o valor”, com a versão nova.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0013-OBSERVABILIDADE-OTEL.MD

Observabilidade com OpenTelemetry, ADOT, CloudWatch e X-Ray

- **Status:** Aceito. A propagação do contexto pela fila está no ADR 0025. A comparação com SaaS e com stack self-hosted está no ADR 0027.
- **Data:** 2026-11-10

Contexto e problema

Um 201 no lançamento e um saldo errado dois segundos depois não cabem no mesmo log se os dois serviços, a fila e o banco não compartilharem um identificador. O pico, a DLQ e a outbox são sintomas operacionais, não só técnicos: fila crescendo é caixa sem relatório.

Drivers

- Os três pilares: log, métrica e trace, correlacionados.
- Funcionar igual no Docker (Grafana LGTM) e na AWS (CloudWatch e X-Ray).
- Métrica de negócio mínima: lançamento registrado, consolidação aplicada, lag.
- Sem agente proprietário que prenda a telemetria a um fornecedor só.

Opções consideradas

1. Log em texto e métrica de CPU do console

- Positivo: zero esforço.
- Negativo: não acha o evento. Não alarma DLQ. Não distingue erro de lançamentos de erro de consolidado. Reprova o diferencial de observabilidade do desafio e, na prática, reprova a operação do RNF.

2. Agente de APM proprietário

- Positivo: dashboard pronto, profiling.
- Negativo: licença, agente fechado, outro formato. A estimativa de custo trata licença de software como zero; um APM comercial quebraria essa premissa sem necessidade neste porte.

3. OpenTelemetry (OTLP) na aplicação, ADOT na AWS, `grafana/otel-lgtm` no local

Micrometer + OTel. Log JSON estruturado com `trace_id` e `span_id`. Actuator: liveness, readiness, Prometheus. Na AWS, sidecar ADOT exporta trace para X-Ray e métrica para CloudWatch. No local, `OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4318` e Grafana em `localhost:3000`.

- Positivo: um SDK, dois destinos. O teste local mostra o trace do POST até o consumidor. Alarmes de 5xx, p99, profundidade da fila, idade da mensagem, DLQ e outbox pendente têm para onde apontar. Sem

licença.

- Negativo: sidecar custa CPU dentro da task (o consolidado reserva 1 vCPU / 2 GiB e o lançamentos 0,5 vCPU / 1 GiB, sidecar incluso). Cardinalidade de label mal escolhida (por exemplo `lancamentoId` como dimensão de métrica) explode a conta do CloudWatch. X-Ray tem preço por trace; amostragem é obrigatória, não opcional. O dashboard de produção está no módulo Terraform de observabilidade. O endpoint de interface do X-Ray existe para o ADOT não precisar de egress aberto.

Decisão

Opção 3. Amostragem de trace em produção começa em 10% das requisições normais e 100% dos erros, ajustável por variável. Log em JSON, nível INFO, sem payload de lançamento completo no log (descrição pode ser dado pessoal; o id basta).

Métricas de domínio, não só RED: contador de lançamentos por tipo, contador de eventos aplicados, gauge de outbox não publicada, lag de consolidação (instante do evento versus instante da aplicação).

Consequências

Positivas

- O runbook de “saldo atrasado” começa pelo lag e pela idade da mensagem, não por uma opinião.
- Trocar o backend de observabilidade no futuro não troca a instrumentação.

Negativas

- ADOT sidecar e a aplicação precisam concordar na propagação de contexto W3C até o atributo da mensagem SNS/SQS. Se o relay não copiar o `traceparent` para a mensagem, o trace quebra no meio e o desenho perde metade do valor. Isso é critério de aceite da instrumentação, não enfeite.
- Custo de log cresce com DEBUG esquecido em produção. O perfil `aws` não habilita DEBUG de framework.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0014-IDEMPOTENCIA.MD

Idempotência na escrita e no consumo

- **Status:** Aceito
- **Data:** 2026-11-10

Contexto e problema

Há duas duplicatas de naturezas diferentes. O cliente do caixa pode repetir o POST porque a resposta se perdeu (rede, timeout do app). O SQS entrega a mesma mensagem mais de uma vez, e o relay pode republicar se cair depois do `Publish` e antes de marcar a outbox. As duas, se tratadas com "INSERT sempre", criam dinheiro que não existiu.

Drivers

- Repetir o POST com a mesma intenção devolve o mesmo lançamento.
- Reentrega do evento não altera o saldo duas vezes.
- A chave do cliente não vaza entre comerciantes.
- Retry com backoff só é seguro se o efeito for idempotente.

Opções consideradas

1. Confiar em "o cliente não repete" e em FIFO com deduplicação de 5 minutos

- Positivo: menos tabela.
- Negativo: timeout no celular repete sim. A janela de deduplicação do FIFO não cobre redrive da DLQ no dia seguinte nem republicação da outbox. E o ADR 0005 não adotou FIFO.

2. Idempotência só no consumidor

- Positivo: protege o saldo, que é o agregado derivado.
- Negativo: o extrato fica com dois lançamentos iguais para uma venda. O caixa, que é a fonte da verdade, estaria errado. O consolidado estaria "certo" em cima de um fato duplicado.

3. Chave no POST e `eventId` no consumo

HTTP. Header opcional `Idempotency-Key` (UUID), único por `(comerciante_id, chave)`, no POST de lançamento e no POST de estorno. A coluna `idempotencia.impressao_digital` guarda o SHA-256 do corpo canônico (no registro: tipo, valor com duas casas, data, descrição e categoria; no estorno: id do lançamento e motivo). A resposta não é armazenada: a repetição relê o lançamento pelo `lancamento_id`.

Situação	Resultado
Primeira vez	201 e persiste

Situação	Resultado
Mesma chave, mesma impressão	200 e o mesmo lançamento
Mesma chave, impressão diferente	409

Sem header, cada POST é um fato novo — o blueprint deixa a chave opcional. Cliente de PDV deve enviá-la; cliente de teste pode omitir.

Consumo. `eventos_processados(event_id primary key)` na mesma transação do `UPSERT` do saldo. Conflito de chave: a mensagem é descartada da fila e o saldo não muda.

- Positivo: as duas pontas do dinheiro ficam protegidas. Retry do relay, do SQS e do cliente deixam de ser incidente.
- Negativo: a obrigação de definir o que é "o mesmo corpo". A impressão cobre o corpo canônico, não o header de autorização. A linha não expira nesta entrega: a mesma chave continua válida enquanto o registro existir. Limpeza é operação, não um TTL de 24 h no código.

Decisão

Opção 3. A idempotência HTTP é por comerciante. Um tenant não bloqueia a chave de outro. O `eventId` é gerado na escrita (UUID) e vai no payload; o consumidor não deduplica por `lancamentoId` apenas, porque o contrato manda deduplicar por `eventId` — na prática os dois são 1:1 hoje, e usar o campo do contrato evita uma segunda regra.

Consequências

Positivas

- O teste de carga pode repetir request sem inventar lançamento.
- Redrive da DLQ é seguro.

Negativas

- 200 na repetição e 201 na criação exigem que o cliente trate os dois como sucesso. Está no RF-04. Cliente que só aceita 201 vai marcar erro numa retentativa que deu certo.
- Hash instável (campo com espaço, escala `150.0` versus `150.00`) gera 409 legítimo e surpreende. A normalização do valor para duas casas acontece antes do hash.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0015-LANCAMENTOS-IMUTAVEIS-ESTORNO.MD

Lançamentos imutáveis, correção por estorno

- **Status:** Aceito
- **Data:** 2026-11-10

Contexto e problema

Caixa editável apaga a história. Um `UPDATE` do valor ou um `DELETE` do lançamento deixa o saldo sem explicação e a auditoria dependente de trigger que alguém pode desligar. O negócio precisa corrigir erro de digitação sem reescrever o fato original.

Drivers

- Trilha reconstruível só com inserts.
- Estorno único. Estorno de estorno proibido — senão a história vira uma corrente ambígua.
- A correção atinge a data de competência original, porque é lá que o saldo do dia ficou errado.
- O consolidado não deve aprender uma segunda linguagem para “desfazer”.

Opções consideradas

1. Update e delete lógicos (`cancelado = true`)

- Positivo: uma linha só, listagem simples.
- Negativo: o fato muda. O evento de integração precisa de um tipo “cancelado” ou o consumidor não fica sabendo. Dois caminhos (criar e cancelar) para um domínio que cabe em “sempre nasce um fato”.

2. Lançamento de ajuste em data de hoje, original intocado sem vínculo

- Positivo: append-only.
- Negativo: o dia original continua errado para sempre. O acumulado até hoje fecha, o relatório do dia do erro não. O comerciante olha o dia, não o acumulado, quando confere o caixa.

3. Novo lançamento de tipo oposto, mesma data, mesmo valor, vínculo `lancamentoEstornadoId`

O original permanece. Índice único parcial impede o segundo estorno. Lançamento que já tem `lancamentoEstornadoId` preenchido não pode ser alvo. O evento publicado é o mesmo `LancamentoRegistrado`, e o consolidado soma o tipo oposto no dia.

O blueprint não tem campo `motivo` no agregado e o POST de estorno não envia `descricao`, embora aceite `{ "motivo": "..." }`. Interpretação desta decisão: a descrição do novo lançamento é o motivo, limitado a 255 caracteres, ou `Estorno do lançamento {id}` se o motivo vier vazio. O motivo não entra no evento v1.

- Positivo: a tabela é a auditoria. O saldo do dia original fecha. Um consumidor só. A proibição de estorno duplo é restrição de banco, então duas requisições paralelas não furam o `if`.
- Negativo: a listagem do dia mostra o original e o estorno, e o saldo "parece" ter mais movimento. É o comportamento honesto; esconder o par na UI é problema de apresentação, fora desta API. Corrigir um estorno feito por engano exige um lançamento novo deliberado, não um meta-estorno — o operador precisa entender isso.

Decisão

Opção 3. Não há endpoint de update nem de delete. Não há evento `LancamentoEstornado`.

Consequências

Positivas

- PITR e backup carregam a história completa sem um log paralelo obrigatório.
- O RF-18 (estorno corrige o dia original) cai direto do modelo, sem job noturno.

Negativas

- Lançamento na competência errada e valor certo se corrige com estorno mais um lançamento novo no dia certo. São dois fatos. O sistema não oferece "mover de data" porque mover seria update.
- A descrição sintética do estorno é uma interpretação documentada aqui, não um campo do blueprint. Se a implementação gravar outra frase, o contrato de API ainda se sustenta; o que não pode ocorrer é persistir o motivo só em log.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0016-REPRESENTACAO-MONETARIA.MD

Representação monetária

- **Status:** Aceito
- **Data:** 2026-11-10

Contexto e problema

Saldo errado por centavo destrói a confiança no consolidado mais rápido do que uma indisponibilidade. O erro clássico é `double`: 0,10 não existe exatamente em binário. O valor atravessa JSON de request, coluna SQL, evento e cache.

Drivers

- Duas casas decimais, meio para cima no sentido bancário usual de escala, sem surpresa de `0.30000000000000004`.
- Moeda única BRL nesta versão.
- Valor sempre positivo no lançamento; o sinal contábil é o tipo (crédito ou débito), não o número.
- Contrato do evento já fixado como string `"150.00"`.

Opções consideradas

1. `double` / `float` de ponta a ponta

- Positivo: menos código.
- Negativo: inaceitável para dinheiro. Rejeitado sem meio-termo.

2. Inteiro de centavos (`15000` = R$ 150,00)

- Positivo: aritmética exata, fácil de somar no evento.
- Negativo: toda borda precisa lembrar a unidade. Um cliente que manda `150.00` pensando em reais grava R$ 1,50 ou R$ 15.000 se alguém converter duas vezes. O blueprint mostra o request em reais (`150.00`) e o evento em string decimal, não em centavos.

3. `BigDecimal` no domínio, `NUMERIC(19,2)` no Postgres, string de duas casas no evento

Value object `Dinheiro` com escala 2, `> 0` e moeda `BRL`. A API aceita número JSON no request, como no exemplo do blueprint (`"valor": 150.00`), e o desserializador usa `BigDecimal`, nunca `double`. A resposta devolve string de duas casas para o cliente não reintroduzir binário ao reexibir. O evento usa string, `toPlainString`, sempre com duas casas. A soma do saldo usa `BigDecimal` e a coluna `NUMERIC`.

- Positivo: o tipo carrega a unidade. O evento é estável entre linguagens (JavaScript que lesse número perderia precisão em valores grandes; string não). `NUMERIC(19,2)` cabe muito além do caixa de um comerciante e rejeita escala ilegal no banco.

- Negativo: o request é número e o evento é string. Essa assimetria está no blueprint e é uma aresta real: um cliente JavaScript que faça `JSON.parse` e reenvie o número pode ter alterado a escala antes de chegar aqui. O serviço normaliza para duas casas ao construir `Dinheiro`. Valor com três casas (`10.125`) é 400, não arredondamento silencioso — arredondar esconderia erro de integração.

Decisão

Opção 3. Proibido `double` em campo de valor, inclusive em teste. Comparação de dinheiro por `compareTo` na mesma escala, não por igualdade de objeto com escala diferente (`10.0` versus `10.00` são o mesmo valor depois da normalização).

Consequências

Positivas

- A soma de muitos lançamentos de centavos fecha com o saldo do dia.
- O contrato do evento não depende de biblioteca decimal do consumidor: é texto.

Negativas

- Três representações (número no POST, numeric no banco, string no evento e na resposta) pedem teste de contrato em cada borda. Sem esse teste, a decisão vira intenção.
- Multi-moeda não cabe neste value object sem trocar a invariante de “sempre BRL”. Quando vier, vem com ADR novo, não com um `if` de moeda espalhado.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0017-API-REST-VERSIONADA-RFC9457.MD

API REST versionada e problemas RFC 9457

- **Status:** Aceito
- **Data:** 2026-11-10

Contexto e problema

O canal do comerciante (e qualquer integração) precisa de um contrato estável para escrever lançamento e ler saldo. Erro hoje, se vier como `{ "message": "erro" }` com HTTP 200 ou com 500 genérico, obriga o cliente a ler texto. A evolução não pode quebrar quem já manda o JSON v1.

Drivers

- Prefixo de versão visível na URL.
- Erro com tipo, título, status e detalhe, `Content-Type: application/problem+json`.
- Documentação gerada do código (springdoc em `/v3/api-docs` e `/swagger-ui.html`).
- O mesmo estilo nos dois serviços.
- Evento com versão própria, independente da versão HTTP.

Opções consideradas

1. Sem versão na URL, evolução “compatível sempre”

- Positivo: URL curta.
- Negativo: a primeira quebra não tem para onde mandar o cliente antigo. Campo opcional resolve adição, não remoção nem mudança de semântica de estorno.

2. Versionamento só por header (`Accept: application/vnd.fluxocaixa.v1+json`)

- Positivo: URL estável, versão rica.
- Negativo: invisível no access log padrão e mais trabalhoso no curl e no WAF por path. Para dois recursos, é cerimônia.

3. `/api/v1` e RFC 9457

Recursos como no blueprint. Erros com `type` relativo (`/problems/valor-invalido` , `/problems/estorno-invalido` , `/problems/idempotencia-conflito` , `/problems/recurso-nao-encontrado`), `title` , `status` , `detail` , `instance` . Corpo de sucesso em `application/json` . OpenAPI via springdoc.

Códigos usados de forma estável:

Situação	HTTP
Token ausente ou inválido, claim ausente	401
Escopo insuficiente	403
Payload inválido, período maior que 366 dias	400
Recurso de outro tenant ou inexistente	404
Estorno repetido, estorno de estorno, mesma `Idempotency-Key` com outro corpo	409
Falha inesperada	500, sem stack no corpo

- Positivo: o cliente trata 409 de forma diferente de 400. O path no ALB casa com a versão. O Swagger é a referência viva do que o código faz.
- Negativo: `type` relativo exige que o cliente não presuma um host fixo. Quebrar o v1 exige manter v1 e v2 juntos por um tempo — custo real de código, e é exatamente o que a versão compra. Problema genérico demais (`about:blank` para tudo) desperdiça o RFC; por isso os `type` acima são fechados e poucos.

Decisão

Opção 3. A versão do evento (`eventVersion`) não segue a versão da URL. Dá para evoluir o JSON do SNS sem abrir `/api/v2` , e o contrário também.

Paginação da lista: `page` a partir de 0, `size` limitado a 50. Resposta com `content` , `page` , `size` , `totalElements` , `totalPages` .

Consequências

Positivas

- O teste de contrato e o smoke assertam status e `content-type` , não uma frase em português.
- WAF e métrica RED separam os dois serviços pelo path, que já carrega o recurso.

Negativas

- Mensagem em `detail` pode mudar sem ser quebra; `type` e `status` não podem. Isso precisa estar dito no OpenAPI para o consumidor não parsear `detail` .
- Dois Swaggers (8081 e 8082). Um portal unificado seria o BFF que não adotamos. Até lá, são duas UIs.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0018-BORDA-ALB-WAF.MD

Borda com ALB e AWS WAF

- **Status:** Substituído (Superseded) pelo ADR 0022
- **Data:** 2026-11-10

O texto abaixo é o registro histórico do ALB público com WAF regional. A borda seguinte é o ADR 0022; o ADR 0031 acrescenta a REST API regional na frente do ALB interno.

Contexto e problema

Os dois serviços precisam de um ponto HTTPS único, com TLS, roteamento por path, health check independente e uma camada que barre abuso antes da task. A autorização de negócio continua no serviço (ADR 0011).

Drivers

- Dois target groups, readiness em `/actuator/health/readiness`.
- TLS com ACM.
- Proteção contra volume abusivo e payload obviamente malicioso, sem cortar o pico legítimo de 50 req/s.
- Mesma aplicação no local, sem a borda AWS — logo a borda não pode ser o único lugar que valida JWT.
- Custo fixo previsível.

Opções consideradas

1. Amazon API Gateway HTTP API na frente, VPC Link até as tasks

- Positivo: throttling por rota, JWT authorizer, plano de uso, se um dia houver API key de parceiro.
- Negativo: hop a mais e cobrança por requisição. O VPC Link não é de graça. O authorizer na borda, se for o único controle, contradiz o ADR 0011. Não temos portal de desenvolvedor nem transformação de payload. Para dois paths estáveis, o Gateway compra recurso que não usamos e cobra por request que o ALB já cobre com LCU.

2. CloudFront + WAF na frente do ALB

- Positivo: borda anycast, cache de GET, WAF no edge.
- Negativo: o GET de saldo é por tenant e não é anônimo; cache na CDN é fácil de errar (vazamento entre comerciantes) e o ganho é pequeno com o Redis já na região. Mais um componente. Pode entrar depois, na frente, sem mudar o serviço, se a latência de clientes distantes de São Paulo doer.

3. ALB público com ACM e AWS WAF

Regras de path:

- `/api/v1/lancamentos*` → lançamentos
- `/api/v1/consolidados*` → consolidado

WAF: managed rules Common, Known Bad Inputs e Amazon IP Reputation, mais uma regra rate-based calibrada acima do pico legítimo de um único cliente. O pico de 50 req/s é do serviço; a regra por IP não pode usar 50 como teto de bloqueio sem medir o fechamento real.

- Positivo: encaixe nativo com ECS, health check por target group (o consolidado doente não tira o lançamento do balanceador), TLS, WAF no mesmo recurso, custo dominado pela hora do ALB (USD 0,034/h em `sa-east-1`) e não pelo request. O serviço local ignora a borda e continua correto porque valida o JWT sozinho.
- Negativo: o ALB não versiona API, não transforma body e não emite API key. Rate-based é instrumento grosso (por IP, janela de 5 minutos). Cliente atrás de NAT compartilhado pode somar vários comerciantes no mesmo IP. Por isso o limite começa folgado e o abuso fino, se aparecer, volta a ser argumento para a opção 1.

Decisão

Opção 3. Actuator que não seja liveness/readiness não é publicado no listener público. HTTP 80 redireciona para 443 ou fica fechado; não há API em claro.

Consequências

Positivas

- Falha de readiness do consolidado não remove as tasks de lançamentos. Isso é o RNF crítico na borda, não só na fila.
- WAF managed rule pega uma classe de abuso sem código na aplicação.

Negativas

- Dois lugares podem dizer 401: nenhum hoje na borda, só o serviço. Se no futuro um authorizer for ligado no ALB, os dois precisam concordar no issuer e nos escopos, senão o diagnóstico vira “funciona no local e não na AWS”.
- Regra rate-based mal calibrada é a forma mais fácil de violar o teto de 5% de perda com tráfego legítimo. Mudança nessa regra exige olhar o k6 e o access log, não só o console.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0019-REGIAO-SA-EAST-1-LGPD.MD

Região sa-east-1 e LGPD

- **Status:** Substituído (Superseded) pelo ADR 0021
- **Data:** 2026-11-10

O texto abaixo é o registro histórico da decisão de residência no Brasil. A decisão vigente é `us-east-1`, com salvaguarda de transferência internacional. Não usar este ADR como fonte da região atual. Os USD 641 e USD 1.214 citados no corpo são da topologia anterior (RDS `db.t4g.small`, ALB público, sem ambiente de hml). A conta vigente está na revisão atual do documento 08.

Contexto e problema

Lançamento de caixa identifica um comerciante e descreve a atividade econômica dele. Pode conter dado pessoal na descrição livre (nome de cliente escrito pelo operador, por exemplo). A região de computação e de armazenamento define se esse dado fica no Brasil e quanto a arquitetura custa.

Drivers

- Residência do dado no Brasil como padrão, não como projeto futuro.
- Minimização: o evento de integração não leva descrição nem categoria.
- Custo comparado com `us-east-1`, explícito, para a escolha não ser só discurso.
- Um comerciante, um claim, sem base legal “inventada” no código: a base da operação do caixa é a execução do serviço pedido pelo próprio comerciante, titular da conta.

Opções consideradas

1. `us-east-1`, mais barato

A mesma topologia custa cerca de **USD 641/mês** contra cerca de **USD 1.214/mês** em São Paulo (premissas e tabela no documento 08), relação de 1,89 vez.

- Positivo: quase metade do custo fixo de NAT, endpoint, RDS e Fargate.
- Negativo: dado de caixa fora do Brasil. Transferência internacional, base legal adicional e um contrato de transferência que este desenho não precisa assumir. Latência maior para um operador no Brasil, secundária perto da questão legal.

2. Multi-região active-passive desde o início

- Positivo: queda de `sa-east-1` deixa de ser o fim do RTO.
- Negativo: replica dado pessoal para outro lugar, dobra custo e abre um desenho de conflito que o volume não pede. Está no roadmap como evolução, com o secundário também no Brasil se a região existir, ou com decisão jurídica explícita se for fora.

3. Tudo em `sa-east-1`

VPC, RDS, ElastiCache, SNS, SQS, Cognito, logs, traces, backups e ECR na região de São Paulo. Sem réplica cross-region. Descrição de lançamento não atravessa o evento. Log de aplicação não imprime descrição nem token.

- Positivo: o dado fica onde o titular está. O desenho é simples. A escolha de custo é consciente: pagamos o prêmio da região para não tratar LGPD como fase 2.
- Negativo: a região é mais cara e tem menos alguns lançamentos de serviço no dia do anúncio global (nada do que usamos está ausente: Fargate, RDS PostgreSQL, ElastiCache, Cognito, WAF). Uma indisponibilidade regional longa não tem failover. O RTO de 1 h do blueprint é intra-região.

Decisão

Opção 3. Backup do RDS na mesma região. Não se copia snapshot para `us-east-1` "por segurança" sem uma análise de transferência — cópia esquecida anula a decisão.

Direitos do titular (acesso, eliminação) não são um endpoint nesta versão: eliminação de lançamento contradiz a trilha imutável. O caminho é processo operacional sobre o comerciante (encerrar a conta, reter o que a obrigação comercial exigir, apagar o que for só cadastro no Cognito). Detalhe no documento de segurança. Não se promete `DELETE` de lançamento para "atender a LGPD" destruindo a contabilidade.

Consequências

Positivas

- A estimativa de custo já nasce na região certa. Ninguém descobre o prêmio de São Paulo na fatura.
- O evento mínimo reduz réplica de dado pessoal no consolidado, no cache e na fila.

Negativas

- Descrição continua no banco de lançamentos e pode conter pessoal. Isso é dado a proteger (criptografia, acesso, minimização de log), não dado a replicar.
- Se a organização já padronizou `us-east-1` para tudo, esta decisão é uma exceção a registrar na governança, não um serviço escondido em outra conta sem dono.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0020-ESTRATEGIA-DE-DEPLOY.MD

Estratégia de deploy

- **Status:** Aceito
- **Data:** 2026-11-10

Contexto e problema

Há dois serviços com SLOs diferentes. Um deploy do consolidado não pode reiniciar o lançamento, e um deploy ruim do lançamento não pode deixar as duas tasks novas no ar recebendo tráfego enquanto as velhas já morreram. O volume não justifica, no primeiro dia, a maquinaria de blue/green com teste sintético no target group espelho.

Drivers

- Rolling com saúde verificada antes de seguir para a próxima task.
- Rollback automático se a task nova não fica pronta.
- Deploy independente por serviço.
- Migração de banco compatível com duas versões da aplicação ao mesmo tempo (a task antiga e a nova convivem durante o rolling).
- Caminho de evolução para blue/green quando o custo de um rollback lento for maior que o custo da ferramenta.

Opções consideradas

1. Recreate (sobe o novo, derruba o velho)

- Positivo: simples, sem duas versões convivendo.
- Negativo: furo de disponibilidade em todo deploy. Incompatível com 99,9% se o deploy for rotina. Também impede expandir o mínimo de duas tasks com segurança.

2. Blue/green com CodeDeploy desde a primeira versão

- Positivo: troca de target group, teste na versão verde, retorno instantâneo para a azul.
- Negativo: duas frotas durante a janela, mais código de pipeline, e o ganho só aparece quando o rollback de rolling (que já existe) for lento demais ou quando quisermos desviar 5% do tráfego. Para o pico atual, é a evolução, não o mínimo.

3. Rolling do ECS com deployment circuit breaker e rollback

`minimumHealthyPercent` 100 e `maximumPercent` 200: a task nova sobe e passa no readiness antes de a antiga sair. Circuit breaker do ECS reverte a deployment se as tasks novas falham. Cada service tem o seu. Flyway só aplica mudança compatível com a versão anterior (coluna nova nullable, sem renomear no mesmo

passo). O passo destrutivo espera todas as tasks estarem na versão nova.

- Positivo: nativo do ECS, sem CodeDeploy para operar no dia 1, rollback sozinho, zero downtime no caso feliz, e o consolidado pode ser implantado sem tocar no service de lançamentos.
- Negativo: o rollback de rolling não é instantâneo: ele inicia outra deployment. Bug lógico que passa no readiness (a task sobe e responde 200 no health, mas calcula saldo errado) não é pego pelo circuit breaker. Isso se pega com teste antes do deploy e com alarme depois, não com o mecanismo de rollout.

Decisão

Opção 3 agora. A opção 2 fica como evolução explícita: quando houver suite sintética pós-deploy (smoke contra o target group verde) e a necessidade de desvio gradual, troca-se o controller de deployment para CodeDeploy blue/green sem mudar a imagem nem o contrato. O gancho é o alarme de 5xx nos cinco minutos seguintes ao deploy, que já existe no documento 07 — hoje ele chama gente; no blue/green ele pode travar a troca.

Migração incompatível não anda no mesmo release que o código que a exige. Dois releases: expande, depois contrai.

Consequências

Positivas

- Deploy vira rotina, não janela. O error budget não é gasto em release.
- O circuit breaker cobre imagem que não sobe, porta errada e readiness que não fica verde.

Negativas

- Readiness que sempre retorna 200 anula a decisão. O readiness do lançamentos verifica o banco; o do consolidado verifica o banco e, de forma degradada e explícita, não transforma Redis fora em "task morta" — senão uma falha de cache derrubaria todas as tasks em sequência. Redis fora é degradação, task segue pronta, métrica de erro do cache dispara.
- Duas versões no ar durante minutos exigem disciplina de Flyway. Um `DROP COLUMN` no mesmo release do código que para de usá-la quebra a task antiga no meio do rolling.

GitOps com ArgoCD, e blue/green com análise, ficam para o dia do EKS (ADR 0028). Não mudam o rolling deste ADR enquanto o scheduler for ECS.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0021-REGIAO-US-EAST-1.MD

Região us-east-1, com transferência internacional

- **Status:** Aceito
- **Data:** 2026-11-10
- **Substitui:** ADR 0019

Contexto e problema

O ADR 0019 fixou `sa-east-1` para o dado de caixa nascer no Brasil. A revisão 2 do blueprint muda a borda para CloudFront com WAF de escopo `CLOUDFRONT` e certificado ACM, e passa a operar três ambientes. WAF e ACM dessa borda são recursos de `us-east-1` mesmo que o banco fique em São Paulo. Manter a computação em `sa-east-1` e a borda em `us-east-1` parte a conta em duas regiões sem resolver residência de verdade: o corpo do POST, que pode trazer descrição com dado pessoal, é inspecionado na borda.

A pergunta deixa de ser “onde é mais barato?” e passa a ser “residência estrita ainda é o requisito, e o que ela custa agora que a borda já toca a Virgínia?”.

Drivers

- Custo comparado com preço de lista, não com ordem de grandeza.
- WAF de escopo CloudFront e o certificado do domínio alternativo do CloudFront exigem `us-east-1`.
- Serviço novo da AWS aparece primeiro na Virgínia. VPC Origin e o ciclo do Advanced JDBC Wrapper são exemplos desta leva, não uma promessa genérica.
- Latência São Paulo → Virgínia na casa de 110–130 ms de RTT, e o que o CloudFront de fato reduz.
- LGPD art. 33: transferência internacional é permitida com salvaguarda, e é proibida se o controlador não conseguir demonstrá-la. Residência não é o único modo de cumprir a lei.
- Um caminho de volta explícito, se um contrato exigir dado só no Brasil.

Preço unitário (Price List, consulta em 2026-11-10)

Item	us-east-1	sa-east-1
Fargate ARM, vCPU-hora	0,03238	0,05570
Fargate ARM, GB-hora	0,00356	0,00612
RDS `db.t4g.medium` Multi-AZ, hora	0,129	0,275
RDS `db.t4g.micro` Single-AZ, hora	0,016	0,034
gp3 Multi-AZ, GB-mês	0,230	0,438

Item	us-east-1	sa-east-1
Valkey `cache.t4g.small`, hora	0,0256	0,0488
NAT, hora e por GB	0,045	0,093
VPC endpoint, hora por AZ	0,010	0,021
ALB, hora	0,0225	0,034
CloudFront, GB, edge América do Sul, primeiros 10 TB	0,110	0,110
CloudFront, GB, edge Estados Unidos, primeiros 10 TB	0,085	—
CloudFront HTTPS América do Sul, 10 mil requests	0,0220	0,0220
EKS control plane, hora	0,10	0,10

A produção inteira, mesmas quantidades, fica em **USD 777/mês** na Virgínia e **USD 1.467/mês** em São Paulo (1,89 vez). Os três ambientes 24×7 diferem em **USD 1.098/mês**, cerca de **USD 13,2 mil/ano**. A conta está no documento 08. CloudFront não entra nessa diferença: o comerciante no Brasil é tarifado pelo edge de São Paulo nos dois desenhos. O API Gateway desta revisão usa o mesmo unitário de request nas duas colunas, então também não entra na diferença.

Não há linha de transferência inter-região no total, porque o desenho não replica dado entre regiões. O custo que a decisão compara é o de morar numa região, não o de copiar bytes para a outra.

Opções consideradas

1. Continuar em `sa-east-1` (ADR 0019)

- Positivo: o disco do RDS, o backup e a fila nascem no Brasil. O texto do ADR 0019 continua válido como descrição desse mundo.
- Negativo: paga-se o prêmio de 1,9 vez. O WAF de CloudFront e o ACM do certificado continuam em `us-east-1`. O corpo do POST atravessa a borda mesmo assim. A "residência" fica incompleta no exato ponto em que a descrição, se o operador escrever dado pessoal, está no request.

2. Multi-região active-passive desde já

- Positivo: queda de uma região deixa de encerrar o RTO.
- Negativo: dobra o piso, replica dado pessoal e exige um único escritor. Continua evolução, não ponto de partida. O RTO de 1 h desta entrega é intra-região.

3. Tudo em `us-east-1`, com salvaguarda de transferência

VPC, RDS, ElastiCache, SNS, SQS, Cognito, logs, traces, backups, ECR, KMS e o ALB interno na Virgínia. CloudFront global, com WAF e ACM em `us-east-1` e PoP em GRU para o comerciante. Sem cópia para `sa-east-1`.

- Positivo: um lugar só. O prêmio de São Paulo sai da fatura. WAF, ACM e dado ficam sob a mesma decisão de transferência, em vez de uma residência no papel e uma borda fora dela. Serviço novo não espera

a região de São Paulo habilitar.

- Negativo: dado de caixa fora do Brasil. O RTT de backbone entra no tempo que o comerciante percebe. Uma indisponibilidade de `us-east-1` não tem failover. A salvaguarda do art. 33 precisa existir de verdade (contrato, minimização, criptografia), não como frase neste ADR.

Decisão

Opção 3. O ADR 0019 fica substituído. Backup e snapshot permanecem em `us-east-1`. Não se copia snapshot para `sa-east-1` "para ter uma cópia no Brasil": essa cópia reabre a transferência no sentido inverso e finge uma residência que o sistema não opera.

Latência

A ordem de grandeza do RTT entre São Paulo e a Virgínia, no backbone, é **110–130 ms**. Não é SLA da AWS; é a faixa usada para a decisão. O CloudFront mitiga o que é handshake, não o que é física:

- TLS 1.2+ termina no edge de GRU. O handshake (dois RTT ou mais) não vai até a Virgínia.
- HTTP/2 e HTTP/3 com conexão persistente: o request seguinte, na conexão já quente, paga **um** RTT de backbone mais o processamento.
- O que não some: esse um RTT. Um POST cujo processamento no ALB leva 80 ms chega ao cliente no Brasil perto de 200 ms com a conexão quente, e bem acima disso na primeira conexão se alguém medir incluindo o TLS até a Virgínia sem edge.

O SLO de p95 < 200 ms do POST continua sendo o tempo **no ALB**, dentro da região, como no documento 05. Redefinir esse SLO como tempo no navegador em São Paulo, com o processamento atual, não fecharia. O caixa aceita a diferença. Um pregão não aceitaria; este produto não é um pregão.

LGPD

O art. 33 da Lei 13.709/2018 permite transferir dado pessoal para fora do Brasil quando o controlador oferece e comprova garantias de cumprimento dos princípios e dos direitos do titular. O mecanismo desta entrega:

- Cláusulas-padrão contratuais e o Data Processing Addendum da AWS, aceitos na conta. AWS é o operador de infraestrutura; o controlador do caixa é o comerciante, e esta solução é operadora perante ele.
- Minimização que já estava no ADR 0019 e continua: o evento não leva descrição nem categoria; o log não leva descrição nem token.
- Criptografia em trânsito e em repouso, CMK, acesso por IAM.
- Registro desta transferência neste ADR e no documento 06.

Dado envolvido: identificador do comerciante, descrição livre (pode conter pessoal por uso, não por contrato), logs de acesso, backups. Não há CPF no contrato.

Se a residência passar a ser exigência (cláusula contratual, norma setorial, ou leitura da ANPD que esta salvaguarda não satisfaça):

1. A computação e o armazenamento voltam para `sa-east-1`. O delta é o da tabela do documento 08: cerca de **USD 690/mês** a mais na produção, cerca de **USD 1.098/mês** nos três ambientes.

2. O WAF de escopo CloudFront continua vendo o corpo do POST. Residência estrita também questiona essa inspeção. A alternativa é WAF regional no ALB em São Paulo e abrir mão do desenho de VPC Origin, de volta à família do ADR 0018, com a borda pública.
3. Snapshot esquecido em `us-east-1` anula a volta. O runbook de saída apaga backup, log e segredo na Virgínia depois da migração, com CloudTrail dessa exclusão.
4. Esta volta é um ADR novo. Não se “reativa” o 0019 por configuração.

Direitos do titular continuam como no ADR 0019: não há `DELETE` de lançamento. Encerrar a conta apaga o cadastro no Cognito e retém a trilha pelo prazo comercial. A transferência internacional não cria um direito de apagar a contabilidade.

Consequências

Positivas

- A estimativa de custo nasce na região em que a fatura vai chegar.
- Borda, segredo e banco obedecem à mesma decisão de lugar.
- O control plane do CloudFront deixa de ser uma exceção escondida numa conta que se dizia “só São Paulo”.

Negativas

- O tempo percebido no Brasil inclui o RTT de backbone. Quem prometer 200 ms no PDV em São Paulo estará prometendo o SLO errado.
- A salvaguarda é contratual e operacional. Sem o DPA aceito na conta e sem a minimização no log, este ADR vira texto.
- Indisponibilidade regional de `us-east-1` continua fora do RTO de 1 h.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0022-CLOUDFRONT-VPC-ORIGIN.MD

Borda CloudFront, WAF e VPC Origin

- **Status:** Aceito, ver ADR 0031
- **Data:** 2026-11-10
- **Atualiza:** ADR 0018, que fica substituído

O texto abaixo é a borda CloudFront + WAF + VPC Origin + ALB interno. O ADR 0031 coloca, por padrão, uma REST API regional entre o CloudFront e o ALB. Com `enable_api_gateway = false`, vale este desenho sem alteração.

Contexto e problema

O ADR 0018 escolheu ALB público com WAF regional. A revisão 2 quer uma única porta de entrada, TLS perto do comerciante no Brasil, o ALB sem IP público, e um lugar onde um front estático futuro caiba sem abrir outra borda. A autorização continua no serviço.

Drivers

- Dois target groups e readiness em `/actuator/health/readiness`. Falha do consolidado não tira o lançamento do balanceador.
- TLS 1.2+ com ACM. O certificado de domínio alternativo do CloudFront mora em `us-east-1`.
- WAF antes da task, sem cortar o pico legítimo de 50 req/s.
- O binário local valida JWT sozinho. A borda não pode ser o único authorizer.
- ALB sem rota a partir da internet.
- Custo fixo previsível. Neste volume a borda inteira fica perto de USD 47/mês em `us-east-1` (custos).

Opções consideradas

1. ALB público e WAF regional (ADR 0018)

- Positivo: um hop a menos, encaixe direto com o target group, WAF na mesma região do ALB.
- Negativo: o ALB tem endereço alcançável. Quem descobre o DNS do balanceador fala com ele sem passar no WAF, a menos que o security group seja tão estreito que na prática já é uma allowlist — e a allowlist de “a internet inteira” não é allowlist. TLS termina na Virgínia (ou em São Paulo, no desenho antigo), então o handshake paga o RTT inteiro. Não há onde pendurar um front estático depois.

2. API Gateway HTTP API, VPC Link até as tasks

- Positivo: throttle por rota, authorizer JWT, plano de uso no dia em que existir parceiro com API key.
- Negativo: cobrança por request, hop a mais, e um authorizer na borda que, se for o único controle, contradiz o ADR 0011. Não há portal de desenvolvedor nem transformação de payload. Para dois paths está-

veis, o Gateway compra recurso que esta API não usa. Continua sendo a opção se um dia houver API key de parceiro e cota por consumidor. Não é o dia 1.

3. CloudFront com WAF de escopo CLOUDFRONT, VPC Origin e ALB interno

Distribuição com `PriceClass_All`, HTTP/2 e HTTP/3, política TLS `TLSv1.2_2021`, certificado ACM em `us-east-1`. WAF: AWSManagedRulesCommonRuleSet, KnownBadInputs, SQLiRuleSet, AmazonIpReputationList, rate-based por IP; AnonymousIpList em modo count no dev. Origem: VPC Origin para o ALB interno, nas subnets privadas. O security group do ALB aceita só o prefix list `com.amazonaws.global.cloudfront.origin-facing`. Sem IP público no ALB.

Comportamento de `/api/*`: `CachingDisabled` e origin request policy `AllViewerExceptHostHeader`.

- Positivo: a internet vê o CloudFront. O ALB não tem rota pública. Shield Standard vem com o CloudFront, sem linha de preço. TLS termina em GRU. A mesma distribuição recebe, depois, um behavior para S3 com Origin Access Control, sem segundo domínio de borda. O WAF de escopo CloudFront é o que a região `us-east-1` já exige, alinhado ao ADR 0021.
- Negativo: mais um componente. O request de API não cacheia, então o CloudFront não reduz carga no ALB; reduz handshake e superfície. O corpo do POST é inspecionado na borda (ver LGPD no ADR 0021). Rate-based continua instrumento grosso, por IP, janela de 5 minutos. Cliente atrás de NAT compartilhado soma vários comerciantes no mesmo IP. O limite começa folgado (a ordem de 30.000 requests por IP a cada 5 minutos, herdada do ADR 0018) e se calibra com access log, não com o número 50 do pico do serviço.

Decisão

Opção 3. O ADR 0018 fica substituído.

Cache desligado em `/api/*` porque a resposta é do comerciante autenticado. Cachear isso na CDN erra de dois jeitos: serve saldo de um tenant a outro se a chave de cache não incluir o token (e incluir o token na chave é um vazamento do token no edge), ou não ganha hit nenhum porque cada token é uma chave. O Redis, dentro da VPC, já é o cache do saldo. A CDN não é o segundo cache dessa API.

Headers de resposta, via Response Headers Policy: `Strict-Transport-Security`, `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`. Não se cacheia, então HSTS é o header que importa de verdade: empurra o cliente para HTTPS nas visitas seguintes.

`/actuator` que não seja liveness/readiness não é regra do ALB. Prometheus fica fora do listener. HTTP na distribuição redireciona para HTTPS. Não há API em claro.

Shield Standard fica ligado pelo fato de a entrada ser CloudFront. Shield Advanced não entra: o piso mensal dele é outra ordem de grandeza, sem um evento de negócio que o peça.

Consequências

Positivas

- Varrer o intervalo público do ALB deixa de ser um caminho. O prefix list é a allowlist.

- Falha de readiness do consolidado continua local ao target group dele. O RNF crítico sobrevive à borda nova.
- O serviço local não muda: valida o JWT com o mesmo código.

Negativas

- Dois lugares podem dizer 403: o WAF e o serviço. 403 de WAF não é 403 de escopo. O access log da distribuição e o log do serviço precisam ser distinguíveis no runbook, senão o erro de budget vira caça ao fantasma.
- Regra rate-based apertada demais é a forma mais fácil de violar o teto de 5% de perda com tráfego legítimo.
- Log de WAF com o corpo do POST guarda descrição. O log redige corpo e o header `Authorization`. Sem essa redação, a minimização do ADR 0021 fura na borda.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0023-RESILIENCE4J-E-VERSOES.MD

Java 25, Spring Boot 4.0.9 e Resilience4j 2.4.0

- **Status:** Aceito
- **Data:** 2026-11-10
- **Atualiza:** ADR 0007, que fica substituído

Contexto e problema

O ADR 0007 escolheu Java 25 e Spring Boot 4 sem pinar o patch, e registrou que biblioteca incompatível com a linha 4 exigiria outro ADR. Na prática o código chegou a andar em Boot 4.1.x, e o starter `resilience4j-spring-boot4` disponível não entrava nesse classpath: ele compila contra `spring-boot-autoconfigure` 4.0.0. O relay ficou só com `proxima_tentativa_em`, sem jitter e sem circuito. O blueprint, na revisão 2, fixa o patch e exige o Resilience4j.

Drivers

- Um patch só, nos dois serviços. Boot 4.0 num e 4.1 no outro não é opção.
- O starter oficial, não um auto-config copiado para o repositório.
- Circuito e retry onde há chamada externa (SNS, Redis), sem colocar essa chamada no caminho do POST.
- Suporte do Boot explícito, com gatilho de saída no dia em que o Resilience4j declarar 4.1.

Matriz

Versões alvo. O agente de integração confere o que o `pom` resolveu (`mvn dependency:tree`). O que esta decisão trava é a linha, não um patch transitivo que o BOM ainda pode mover dentro da linha.

Peça	Linha
Java	25 (LTS)
Spring Boot	**4.0.9**, última 4.0.x desta decisão
Spring Framework	7.0, trazido pelo BOM do Boot 4.0
Resilience4j	**2.4.0**, artefato `resilience4j-spring-boot4`, mais `spring-boot-starter-aop` (AspectJ), como o Boot 4.0 exige para o starter
Jackson	3, trazido pelo Boot 4.0 (`tools.jackson`)
Hibernate	7, no serviço que usa JPA (lançamentos)
Testcontainers	2, artefatos `testcontainers-junit-jupiter` e `testcontainers-postgresql`
AWS SDK for Java	v2 (pin observado no pom em 2026-11-10: 2.57.12; o patch pode ter sido ajustado pelo agente de serviço)

Peça	Linha
AWS Advanced JDBC Wrapper	linha compatível com o driver PostgreSQL usado pelo Boot 4.0; o plugin `awsSecretsManager` é o requisito, a versão exata é a do pom

O único release do `resilience4j-spring-boot4` na 2.4.0 compila contra autoconfigure 4.0.0 / `resilience4j-spring6`. Boot 4.1 não tem suporte declarado. Subir o parent para 4.1 com esse starter no classpath é build quebrado ou, pior, auto-config que não dispara. Copiar o auto-config para dentro do serviço seria um fork. Os dois foram recusados.

Onde cada padrão entra

Padrão	Onde	Para quê
CircuitBreaker + Retry (backoff exponencial com jitter) + TimeLimiter	Relay, cliente SNS	Falha do publish não vira loop. Circuito aberto adia o lote; o evento fica na outbox.
CircuitBreaker + timeout, fallback no Postgres	Consolidado, cliente Redis	Cache lento ou morto não segura o GET nem derruba a readiness.
Bulkhead (semaphore)	Consumidor SQS	Limita o paralelismo da task para o pool JDBC continuar sendo o teto de propósito, não de acidente.
Retry com backoff	Consumidor, falha transitória de banco	Dentro do visibility. Não retenta erro de domínio.
RateLimiter	Consultas do consolidado, **opcional** e desligado	Só se um cliente legítimo, atrás de um IP que o WAF não separa, passar a ameaçar o p99. Ligado cedo demais, vira a causa da perda de 5%.

Métricas do Resilience4j saem no Micrometer e no `/actuator/prometheus`. O estado do circuito aparece em `/actuator/health` sem mudar a readiness para `DOWN`. Não existe circuito entre os dois serviços: não existe chamada entre eles.

O backoff da coluna `proxima_tentativa_em` continua. O Resilience4j não substitui a outbox; ele impede que o relay, com o SNS doente, publique em rajada no instante em que o SNS volta.

Ciclo de suporte e gatilho de upgrade

A política publicada pela Spring para o Boot dá à minor uma janela de suporte open source e, depois, suporte comercial. A página de suporte do projeto é a fonte da data: esta decisão não inventa o dia do GA. Se a janela open source da 4.0 estiver no fim, isso **não** autoriza pular para 4.1 enquanto o starter não declarar compatibilidade. Ficar na última patch 4.0.9, com o CVE acompanhado, é preferível a um classpath que o Resilience4j não suporta.

Gatilho. O primeiro artefato `resilience4j-spring-boot4` cuja POM compila contra `spring-boot-autoconfigure` 4.1.x. No dia em que isso existir:

1. Os dois poms sobem juntos, na mesma pull request.
2. Rodam ArchUnit, Testcontainers e o teste de contrato do evento.

3. Não se mantém um serviço na 4.0 e o outro na 4.1.

Até lá, dependabot que proponha Boot 4.1 é recusado com ponteiro para este ADR.

Opções consideradas

1. Boot 4.1 sem Resilience4j

Era o estado que o ADR 0007 acabou descrevendo na implementação. O relay só com timestamp. Sem jitter, duas tasks acordam juntas. Sem circuito, um SNS lento ocupa o agendador até o timeout do cliente HTTP.

2. Boot 4.1 com auto-config forkado

- Positivo: “temos circuito” e o parent novo.
- Negativo: o fork não recebe o patch do Resilience4j. O dia em que o starter oficial sair para 4.1, o fork briga com ele.

3. Boot 4.0.9 e Resilience4j 2.4.0

- Positivo: o starter oficial. Os dois serviços na mesma patch. O circuito fica onde há I/O externo.
- Negativo: a linha 4.0 envelhece enquanto a 4.1 segue. Aceito, com o gatilho acima. Virtual threads e o teto do Hikari continuam como no ADR 0007: thread barata não autoriza pool infinito.

Decisão

Opção 3. Java 25, Temurin, imagem não-root, um Maven wrapper por serviço. Pacotes `br.com.fluxocaixa.lancamentos` e `br.com.fluxocaixa.consolidado`.

Consequências

Positivas

- O POST continua sem SNS no caminho, e o relay ganha um limite que o timestamp sozinho não expressa.
- A matriz cabe numa página. Surpresa de versão vira diff de pom, não virada de sprint.

Negativas

- Ficar na 4.0 é dívida com data. O gatilho precisa de um dono (a revisão mensal do release note do Resilience4j), senão a dívida vence em silêncio.
- RateLimiter opcional é uma tentação. O default é desligado.

Rotação de segredos

- **Status:** Aceito
- **Data:** 2026-11-10

Contexto e problema

Senha de banco em variável de ambiente, gravada na task definition, não gira. Quando gira à mão, o pool JDBC segue com a senha velha até alguém reiniciar a task, e o intervalo entre as duas ações é uma janela de 500. O blueprint exige rotação automática no RDS, releitura pelo Advanced JDBC Wrapper, e o mesmo tratamento para KMS, ACM, JWKS e a credencial de CI.

Drivers

- Segredo nenhum no Git e nenhum na imagem.
- Rotação que a aplicação sobrevive sem deploy.
- CI sem access key de longa duração.
- Alarme quando a rotação falha, não quando o cliente já está tomando 500.
- Lista fechada do que é segredo, com dono e frequência. O que não está na lista não é “segredo informal”.

Opções consideradas

1. Senha no Secrets Manager, lida só na subida da task

- Positivo: sai do Git. A task definition aponta o ARN.
- Negativo: rotação deixa a task com a senha velha na memória do pool. O conserto é redeploy, e quem gira o segredo precisa lembrar do redeploy. Esqueceu, o RDS aceita a senha nova e a task usa a antiga.

2. RDS IAM authentication

- Positivo: token curto, sem senha de longa duração.
- Negativo: o token dura 15 minutos e o pool precisa de um plugin que o renove. É outro mecanismo, válido, e não é o que o blueprint padronizou. Fica como alternativa se o wrapper não se sustentar. Não é a decisão.

3. Senha gerenciada pelo RDS, rotação no Secrets Manager, wrapper que rebusca

`manage_master_user_password` com rotação automática. O contrato da task é `DB_URL=jdbc:postgresql://...`. Com `DB_SECRET_ARN` o serviço normaliza para um único `jdbc:aws-wrapper:postgresql://...` e liga `failover,awsSecretsManager`. No endpoint do RDS Proxy (hostname

com `.proxy-`) o `failover` sai e fica só `awsSecretsManager`: o plugin iria na instância e furaria o proxy. Ausente o segredo (local e teste), o driver PostgreSQL comum usa `DB_USERNAME` e `DB_PASSWORD`, mesmo que a URL tenha vindo com o prefixo do wrapper.

Hikari `maxLifetime` de 15 minutos, menor que o intervalo de rotação. A role da task tem `secretsmanager:GetSecretValue` só no ARN do próprio segredo e `kms:Decrypt` na CMK dele.

- Positivo: a rotação não pede deploy. Falha de autenticação faz o wrapper ler o segredo de novo. O failover de Multi-AZ usa o mesmo wrapper, em vez de um pool cego ao DNS novo.
- Negativo: mais uma dependência no classpath. Local não exerce esse caminho; o teste que importa é de integração com o wrapper apontando para um segredo de mentira, mais o ensaio de rotação em dev. Sem esse ensaio, o plugin é configuração não verificada.

Decisão

Opção 3. Frequência: **30 dias** em hml e prod, **90 dias** em dev. Dev gira menos porque o ambiente é descartável e a rotação semanal só acrescenta ruído; 90 dias ainda impede senha eterna.

O restante gira pelo mecanismo do serviço, não por um cron nosso:

Segredo ou credencial	Dono	Mecanismo	Frequência
Senha do RDS de lançamentos	RDS + Secrets Manager	Rotação gerenciada, wrapper rebusca	30 dias (prod, hml); 90 (dev)
Senha do RDS de consolidado	idem	idem	idem
CMK (RDS, fila, log, segredo)	KMS	Rotação automática da CMK	anual
Certificado ACM do CloudFront	ACM	Renovação gerenciada, certificado em `us-east-1`	gerenciada; alarme se faltar menos de 14 dias e a renovação não tiver ocorrido
Chave de assinatura do JWT (Cognito)	Cognito	JWKS buscado pelo serviço, cache curto	quando o Cognito girar; o serviço não guarda PEM
Credencial de CI	GitHub OIDC → role IAM	Sem access key. Trust policy com o repositório e o environment	a role não tem segredo para girar
Token do Redis	não existe nesta versão	TLS sem AUTH	—
Senha do Grafana local	imagem `otel-lgtm`	`admin` / `admin`, só no Compose	nunca em produção, porque esse Grafana não vai para a AWS
Usuário do Keycloak local	realm de laboratório	senha de laboratório no Compose	não tem correspondente na AWS

AUTH token do ElastiCache fica desligado. Ligar cria o terceiro segredo (USD 0,40/mês) e um segundo wrapper de credencial no caminho do GET. Vale quando o security group deixar de ser a fronteira suficiente — por exemplo, subnet de dados compartilhada com outra aplicação. Até lá, TLS mais security group. A decisão de

ligar é consciente, registrada aqui para não parecer esquecimento.

CI: GitHub Actions assume a role com `id-token`. Não há `AWS_ACCESS_KEY_ID` em secret do repositório. Environment `prod` exige aprovação. A role de CI de aplicação não tem `rds:DeleteDBInstance`. A role de Terraform é outra, também via OIDC, presa ao workflow de infra.

Alarme: EventBridge em `RotationFailed` do Secrets Manager, no tópico operacional. O alarme dispara na falha da rotação, não no 500 do dia seguinte.

Runbook de rotação manual ou emergencial

Quando a rotação automática falhar, ou quando uma senha tiver vazado:

1. Confirmar o alarme e o segredo (qual ARN, qual serviço). Não girar os dois "por garantia" sem olhar o log: o segundo giro no meio do primeiro confunde o wrapper.
2. `aws secretsmanager rotate-secret` no ARN, ou a rotação imediata no console, se a lambda de rotação estiver saudável.
3. Se a lambda estiver quebrada: colocar a senha nova com `PutSecretValue`, atualizar o usuário no RDS para a mesma senha, e só então observar as tasks. O wrapper rebusca quando a autenticação falha. Se o pool não soltar a conexão velha, `maxLifetime` de 15 minutos limita a espera; forçar novo deployment do serviço ECS corta essa espera.
4. Verificar conexão nova no Performance Insights ou em `pg_stat_activity` (usuário da aplicação, timestamp recente) e o 5xx do serviço.
5. Se a senha vazou para log ou ticket, tratar como incidente: girar, invalidar o que foi exposto, e corrigir o log. Girar sem corrigir o log repete o vazamento no próximo stack trace.

KMS com rotação anual não exige esse runbook: a chave antiga continua decriptando o ciphertext já gravado. Não se desabilita a CMK "para forçar". Desabilitar a CMK do RDS é indisponibilidade, não rotação.

Consequências

Positivas

- Vazamento da senha de hoje tem prazo. O prazo é 30 dias no máximo, e minutos se o runbook rodar.
- O mesmo binário sobe local sem o wrapper, porque `DB_SECRET_ARN` vazio cai no driver comum.

Negativas

- O caminho de produção do JDBC não é o caminho do Testcontainers. O ensaio de rotação em dev é obrigatório, uma vez por trimestre, junto do game day.
- Dois mecanismos (wrapper e, se um dia ligarmos, Proxy) que ambos falam de conexão. Proxy não substitui o wrapper para segredo, e o wrapper não multiplexa conexão. Ligar os dois sem essa frase vira configuração duplicada.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0025-RASTREABILIDADE-CONTEXTO.MD

Rastreabilidade e propagação de contexto

- **Status:** Aceito
- **Data:** 2026-11-10
- **Complementa:** ADR 0013

Contexto e problema

O ADR 0013 escolheu OpenTelemetry, ADOT, CloudWatch e X-Ray. O trace do POST morria no commit: o relay roda noutra thread, o SNS não carrega o contexto W3C sozinho, e o consumidor abria um trace novo. O lag de consolidação virava "dois sistemas" no Grafana. Suporte com um id na mão não achava a linha do outro serviço.

Drivers

- Um trace só, do POST ao `UPSERT` do saldo, também no X-Ray.
- Um identificador que o cliente possa guardar e devolver, sem ser o trace id (o cliente não tem obrigação de falar W3C).
- O contrato JSON do evento estável. Telemetria não entra no corpo.
- Log JSON em que o campo exista de verdade, com exemplo, não só o nome da chave num diagrama.

Opções consideradas

1. Só `eventId` no log

- Positivo: zero propagação. O id de negócio já existe.
- Negativo: o `eventId` nasce no commit. O tempo de JWT, a ida ao banco e a recusa 409 não têm `eventId`. O cliente não recebe esse id antes do 201. Não dá para correlacionar o 401.

2. `traceparent` dentro do JSON do evento

- Positivo: um campo só, o consumidor que já parseia o corpo vê o contexto.
- Negativo: toda versão do contrato e todo consumidor de BI passam a carregar telemetria. O JSON deixa de ser só fato contábil. A raw delivery continua, mas o payload inchou com dado que não é linguagem publicada.

3. W3C no HTTP, coluna na outbox, atributo na mensagem

`traceparent` e `tracestate` entram e saem no HTTP pelo Micrometer Tracing com a ponte OpenTelemetry. `X-Correlation-Id` é aceito se for UUID; se vier ausente ou inválido, o serviço gera outro, devolve no response e guarda no MDC como `correlationId`.

No commit, a linha da outbox grava `trace_parent`, `trace_state` e `correlation_id`. O relay, noutra thread, abre um span `PRODUCER` filho desse pai e publica os atributos `traceparent`, `tracestate` (se houver), `correlationId` e `eventType`, todos `String`. O corpo não muda.

O consumidor pede os atributos (`MessageAttributeNames=All`), abre um span `CONSUMER` filho do produtor, preenche o MDC e limpa no fim. Log JSON leva `traceId`, `spanId`, `correlationId` e, quando existirem, `comercianteId`, `lancamentoId`, `eventId`.

- Positivo: o contrato da seção 6 do blueprint permanece. O trace atravessa a fronteira assíncrona. O cliente guarda um UUID, não um header de trace.
- Negativo: esquecer `MessageAttributeNames=All` quebra a propagação em silêncio (o corpo chega, o contexto não). Isso é teste de contrato, não convenção oral. O formato do trace no X-Ray não é o hex de 32 caracteres do log; o documento 07 diz como navegar.

Decisão

Opção 3. Amostragem segue a do ADR 0013 (cabeça no ingress, 10% em produção como ponto de partida). O filho herda a decisão de amostragem do pai: um POST amostrado produz o consumidor amostrado, e um POST não amostrado não acende o consumidor. Amostrar os dois lados de forma independente faria o trace "sumir no meio" justamente nos pedidos que se queria ver.

`comercianteId` é atributo de span e campo de log. Não é label de métrica.

Consequências

Positivas

- Um id devolvido no 201 alcança o log do consolidado.
- Dá para distinguir falha de publish (span produtor sem filho consumidor) de falha de projeção (filho consumidor com erro).

Negativas

- Três ids na mesa (`correlationId`, `traceId`, `eventId`). O documento de integração fixa o papel de cada um. Usar um no lugar do outro é bug de suporte.
- A coluna nova na outbox é obrigatória no insert do POST. Relay que publica sem reler a coluna recria o trace quebrado que este ADR existe para acabar.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0026-ECS-AGORA-EKS-EVOLUCAO.MD

ECS agora, EKS como evolução

- **Status:** Aceito
- **Data:** 2026-11-10
- **Complementa:** ADR 0009, que permanece aceito

Contexto e problema

O ADR 0009 colocou os dois serviços em ECS Fargate. A pergunta volta porque a comparação com Kubernetes é o que uma banca espera ver respondida com número, não com preferência. EKS tem control plane, ecossistema e uma stack que o ECS não pede. Também tem um piso.

Drivers

- Dois serviços, um time, pico que cabe em quatro tasks.
- Custo de control plane explícito: USD 0,10/hora, **USD 73/mês** por cluster, nas duas regiões (Price List, 2026-11-10).
- Operação que o time consegue fazer sem um time de plataforma.
- Porta de saída no dia em que o número de serviços ou de times mudar o desenho.

Comparação

Eixo	ECS Fargate	EKS
Control plane	incluso no preço da task	USD 73/mês por cluster, mesmo vazio
Onde o processo roda	task, sem nó	nó (ou Fargate profile, que devolve o preço de Fargate e tira o bin packing)
Produção deste projeto	USD 777/mês na conta base	USD 950/mês com 3 × `m7g.large`, mesma rede e mesmos dados (custos)
Operação	service, task definition, target group	nó, NodePool, add-on, versão do control plane, upgrade com extended support se atrasar
Curva	o time que já escreve o Terraform de ECS opera o deploy	pede quem já operou Kubernetes, ou tempo para aprender no sistema de caixa
Ecossistema	o da AWS (autoscaling de service, circuit breaker de deploy)	o da CNCF, listado no documento 13
Portabilidade	imagem OCI, que também roda no EKS	manifesto Kubernetes, que não roda no ECS
Bin packing	a task reserva 0,5 ou 1 vCPU o mês inteiro, ociosa ou não	vários processos no mesmo nó, com o desperdício só da folga do nó

Eixo	ECS Fargate	EKS
Autoscaling	CPU, request do ALB, backlog da fila, nativo	HPA, e KEDA para a fila; Karpenter para o nó
Service mesh	não há, e não falta: os serviços não se chamam	Istio ou Linkerd quando houver chamada síncrona interna
GitOps	o pipeline faz push do novo task definition	ArgoCD puxa o manifesto (ADR 0028)

A densidade só paga o control plane quando há o que empacotar. Quatro tasks pequenas em três nós `m7g.large` deixam nó sobrando e ainda pagam USD 73. O bin packing começa a favorecer o EKS quando dezenas de processos dividem o nó. A conta de 30 serviços está no documento 13.

Opções consideradas

1. EKS agora

- Positivo: o manifesto nasce no formato da evolução. Não há migração depois.
- Negativo: USD 73 mais três nós (USD 179) no lugar de USD 87 de Fargate, mais a stack se alguém "já que tem cluster" instalar Istio e Prometheus. Dois serviços não usam o que o cluster oferece. O risco é operar a plataforma em vez do caixa.

2. ECS agora, EKS quando o gatilho disparar

- Positivo: a fatura e a operação cabem no time. A imagem OCI não se joga fora na migração: o que muda é o scheduler e o deploy.
- Negativo: o dia da migração existe. Task definition não vira Deployment sozinha. O documento 13 é o mapa dessa migração, para ela não ser inventada na pressa.

3. ECS em todo o horizonte

- Positivo: nunca se paga control plane.
- Negativo: trava a resposta. Com muitos times, o ECS sem uma plataforma interna vira um Terraform por time e um pipeline por serviço, que é uma plataforma pior, só que sem o nome.

Decisão

Opção 2. ECS Fargate ARM, como no ADR 0009, para os dois serviços.

EKS passa a valer quando a maioria destes for verdadeira:

- mais de **8 a 10** serviços deployáveis, ou mais de um time publicando com calendário próprio;
- necessidade de plataforma interna (template de serviço, política de admissão, GitOps) que o time aceite operar;
- bin packing ou escala por fila (KEDA) que o autoscaling do ECS já não expresse sem contorção.

Um desses sozinho não basta. Dez serviços de um time só ainda podem viver em ECS. Dois serviços com vontade de Istio não justificam o cluster.

Até o gatilho, não se cria “um EKS pequeno para ir aprendendo” em prod. Aprender é cluster efêmero, destruído, fora da conta do caixa.

Consequências

Positivas

- A decisão de hoje tem preço (USD 777 contra USD 950) e um número de serviços que a reabre.
- A stack open source fica especificada antes de alguém instalar o chart da moda. Especificada não é instalada.

Negativas

- Quem comparar só o ecossistema, sem a fatura e sem o tamanho do time, vai achar a decisão tímida. O documento 13 existe para essa conversa ter os dois lados.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0027-OBSERVABILIDADE-OSS-VS-SAAS.MD

Observabilidade: gerenciada agora, SaaS ou self-hosted depois

- **Status:** Aceito
- **Data:** 2026-11-10
- **Complementa:** ADR 0013

Contexto e problema

O sinal é OpenTelemetry nos dois desenhos (local e AWS). O destino é que muda o custo e o plantão. CloudWatch e X-Ray já estão no caminho do ADOT. Grafana Cloud, Datadog, New Relic, Dynatrace e uma stack self-hosted no EKS resolvem o mesmo problema com outra fatura e outra pessoa de plantão. O volume de hoje é pequeno; a escolha errada ou é um piso de SaaS ocioso ou é um Prometheus que ninguém atualiza.

Drivers

- Um destino só para log, métrica e trace, com correlação (ADR 0025).
- Custo no volume real (4 tasks, ~20 GB de log/mês, ~2.000 séries) e no volume de 30 serviços.
- Custo de gente na conta. Premissa de planejamento: USD 8.000/mês carregados por engenheiro sênior.
- Não operar Loki sem o time que já opera cluster.

Opções consideradas

1. ADOT, CloudWatch e X-Ray

É o ADR 0013. Cerca de USD 45/mês na estimativa de produção.

- Positivo: sem contrato novo, sem agente extra na imagem, alarme no mesmo SNS operacional. O trace já tem caminho para o X-Ray.
- Negativo: PromQL não é a língua do CloudWatch. Métrica customizada a USD 0,30 por métrica/mês pune cardinalidade. Log acima de ~100 GB/mês, a USD 0,50/GB de ingestão em `us-east-1`, deixa de ser “incluso no ruído”.

2. SaaS (Grafana Cloud, Datadog, New Relic, Dynatrace)

No volume deste projeto, a lista de 2026-11-10 fica entre cerca de USD 19 (Grafana Cloud Pro, dentro da franquia) e cerca de USD 52 (Datadog no SKU serverless). Números e premissas no documento 13 e no documento 08.

- Positivo: alguém atualiza o armazenamento. Correlação de log e trace é o produto. Grafana Cloud é o salto menor, porque o local já é Grafana.

- Negativo: dado de telemetria (com `comercianteId`) sai para outro operador. É outra transferência, com o DPA daquele fornecedor, não coberto pelo DPA da AWS. O contrato precede o agente.

3. Stack self-hosted (Prometheus ou Mimir, Loki, Tempo, Grafana)

Cerca de USD 136/mês de nó e disco no cenário do documento 08, mais fração de pessoa.

- Positivo: o dado de telemetria fica na conta. A língua é a mesma do laboratório.
- Negativo: USD 2.000 de um quarto de engenheiro superam USD 136 de nó e superam o SaaS. Sem o plantão, a stack cai e o caixa fica cego — pior do que um CloudWatch sem PromQL.

Decisão

Opção 1, agora.

Grafana Cloud é a primeira alternativa no dia em que o time quiser PromQL ou o log passar de ~100 GB/mês, **e** o DPA do fornecedor estiver aceito. Datadog, New Relic e Dynatrace entram na mesma conversa quando o valor pedido for APM com perfil ou análise de causa, não quando o valor pedido for "ver o trace": isso o X-Ray e o Tempo local já fazem. A comparação de 30 serviços está no documento 13 para essa conversa não usar o preço de 4 tasks.

Self-hosted entra junto com o EKS, quando o ADR 0026 disparar e já existir quem atualize o cluster. Instalar a stack antes do time é adotá-la duas vezes: uma agora, outra quando o cluster existir.

Consequências

Positivas

- Um destino, já instrumentado. O dinheiro de observabilidade neste volume é ruído perto de NAT e RDS.
- A troca futura não mexe no código de negócio: o SDK é OpenTelemetry. Muda o exporter.

Negativas

- Investigação rica (PromQL, exemplars) fica mais pobre no CloudWatch do que no Grafana. Aceito no tamanho atual.
- A estimativa de USD 45 é ordem de grandeza. A primeira fatura calibra. Se vier o dobro, a conversa de Grafana Cloud antecipa, sem esperar os 30 serviços.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0028-GITOPS-ARGOCD-EVOLUCAO.MD

GitOps com ArgoCD, como evolução

- **Status:** Aceito
- **Data:** 2026-11-10
- **Complementa:** ADR 0020

Contexto e problema

O deploy de hoje é push: o GitHub Actions assume uma role e chama `UpdateService` no ECS, com a imagem que o CI já construiu. Funciona com dois serviços. Deixa de funcionar como modelo quando o que se implanta é um conjunto de manifestos (quando o EKS existir) ou quando o que está rodando diverge do que está no Git e ninguém vê.

Drivers

- Build once, já decidido: a mesma imagem promove dev → hml → prod.
- O Git como fonte do que deveria estar rodando.
- Drift visível.
- Não operar ArgoCD antes de existir o cluster que ele gerencia.

Opções consideradas

1. Push pelo Actions, para sempre

- Positivo: o pipeline atual. Um log, uma role, um `UpdateService`.
- Negativo: o estado desejado fica repartido entre o Terraform (a task definition) e o workflow (a tag da imagem). Os dois podem divergir, e o próprio comentário do workflow de CD já avisa: o próximo `terraform apply` devolve a tag antiga se ela não tiver sido alinhada. Isso é drift com nome.

2. ArgoCD agora, em ECS

- Positivo: um só modelo mental.
- Negativo: ArgoCD não é o controlador nativo de service ECS. Forçar GitOps em ECS antes do EKS cria um controlador terceiro para um recurso que o Terraform e o Actions já cobrem. Dois donos do service.

3. Push no ECS agora; ArgoCD quando houver EKS

Repositório de manifests (ou uma pasta por ambiente), ArgoCD em modo pull, app-of-apps, ApplicationSet por serviço. A imagem promovida é um commit de tag nesse repositório, não um `kubectl` na workstation. Drift detection ligado: o que mudou no cluster fora do Git aparece como diff, não como “ajuste”.

- Positivo: o cluster não aceita o que o Git não diz. O mesmo mecanismo serve aos add-ons da stack (Karpenter, External Secrets, cert-manager), senão a plataforma vira clique.

- Negativo: mais um componente de alta disponibilidade. ArgoCD fora do ar não tira o caixa do ar (o que já está implantado continua), mas trava o deploy. Precisa de dono.

Decisão

Opção 3. Enquanto o scheduler for ECS, o deploy continua o do ADR 0020: rolling, circuit breaker, a tag imutável publicada pelo CI, promoção dev → hml → prod com aprovação no environment. O alinhamento da tag com o Terraform é dever do pipeline, não um lembrete no comentário do YAML.

Quando o ADR 0026 disparar, o GitOps entra no mesmo passo do cluster, não "na fase 2 do EKS". Cluster sem ArgoCD e com apply manual é o modo mais rápido de a stack open source divergir do documento.

Pull, não push: o cluster busca o Git. A role de CI não precisa de privilégio de `kubectl apply` em prod. A role do ArgoCD precisa, e fica no cluster, com a trilha de quem aprovou o commit.

Consequências

Positivas

- A evolução tem repositório, direção do fluxo e o que fazer com drift, antes de a ferramenta existir.
- Build once não muda com a ferramenta. Muda o ato de implantar.

Negativas

- Até lá, o drift Terraform versus tag do ECS é um risco real. O documento de CI/CD trata esse ponto no pipeline atual, para não esperar o ArgoCD resolver um problema que o ECS já tem.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0029-TRES-AMBIENTES-STATE-REMOTO.MD

Três ambientes e state remoto do Terraform

- **Status:** Aceito
- **Data:** 2026-11-10
- **Complementa:** ADR 0010, que permanece aceito

Contexto e problema

O ADR 0010 escolheu Terraform com ambientes `dev` e `prod` e state remoto. A revisão 2 acrescenta `hml`, com tamanho próprio (seção 15 do blueprint): hml ensaia a produção sem pagar três NAT e sem Multi-AZ. Dois ambientes obrigavam o ensaio a acontecer em dev (barato demais, rede diferente) ou em prod (tarde demais).

Drivers

- Um state por ambiente, lock, sem `tfstate` no Git.
- Promoção de imagem dev → hml → prod, com a infra de cada um já existente. Promover infra "junto com a imagem" no mesmo apply é como o dev de um experimento derruba a rede de prod.
- Hml parecido com prod no caminho da aplicação (endpoints, Container Insights, on-demand) e menor na rede e no banco.
- Apply de prod só pelo pipeline, com aprovação.

Opções consideradas

1. Dev e prod, hml "quando precisar"

- Positivo: uma conta a menos de NAT e de RDS.
- Negativo: o primeiro ensaio de migração Flyway com duas versões da aplicação acontece em prod. O custo de hml (USD 356/mês em `us-east-1` se ficar ligado o mês inteiro) é o preço de não ensaiar em prod.

2. Três contas AWS, uma por ambiente

- Positivo: explosão de raio. Uma credencial de dev não vê prod.
- Negativo: para dois serviços, o custo de operar três contas (trail, usuário, limite) compete com o benefício. A separação que importa neste porte é state, role e approval, não a conta. Conta separada continua evolução se o produto ganhar outro sistema na mesma organização.

3. Três ambientes, três states, na mesma conta, em S3

Prefixo por ambiente no bucket de state, lock nativo do S3 (`use_lockfile`), criptografia com KMS, versionamento do bucket. Um módulo, três `tfvars` . Nomes de fila e tópico com o prefixo do ambiente, como o blueprint já exige (`fluxo-caixa-dev-` , `fluxo-caixa-hml-` , `fluxo-caixa-prod-`). A aplicação continua lendo ARN e URL, não o nome.

- Positivo: `plan` de hml não é `plan` de prod. O lock impede dois applies no mesmo state. O versionamento do bucket é o rollback do state, não do RDS.
- Negativo: a mesma conta significa que uma role mal escrita enxerga os dois RDS. A fronteira é IAM, e IAM errado não tem a rede de segurança da conta separada. A role de dev não tem permissão no state de prod. Isso é política, testada no pipeline, não só escrita.

Decisão

Opção 3. Backend S3 em `us-east-1` , junto da infra. Não há state em `sa-east-1` .

Dimensionamento é o da seção 15 do blueprint, repetido nos tfvars, não copiado para este ADR para não virar uma terceira fonte. O que este ADR fixa é a existência dos três e a regra de cada um:

Ambiente	Para quê	O que não é
dev	Integração na AWS, Spot, Single-AZ, sem endpoint	Produção pequena
hml	Ensaio de migração, de rotação de segredo e de deploy, com endpoints e sem Multi-AZ	Cópia fiel da fatura de prod
prod	Tráfego do comerciante, Multi-AZ, deletion protection, três NAT	Lugar de experimentar tfvars

Destroy de prod não é alvo do dia a dia. Destroy de dev é permitido e é o jeito certo de parar a fatura, melhor do que “desligar task” e deixar NAT.

O pipeline de Terraform (`fmt` , `validate` , `tflint` , Checkov, e o que o documento 14 acrescenta) roda nos três. Apply automático, se existir, para em dev. Hml e prod exigem aprovação do environment.

Consequências

Positivas

- O ensaio de Flyway com rolling tem um lugar que não é o caixa do comerciante.
- A fatura de dev (USD 143) e a de hml (USD 356) estão na estimativa antes de alguém criar “rapidinho”.

Negativas

- Três ambientes ligados 24×7 custam USD 1.216/mês antes de qualquer otimização de horário. Hml desligado à noite economiza task, não endpoint. A decisão de manter hml ligado é operacional e se revê na primeira fatura, com o número na mesa.

- State remoto é agora um componente crítico. Perder o bucket sem versionamento é perder a capacidade de mudar a infra com segurança. Versionamento e a CMK fazem parte do ambiente, não do “depois”.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0030-REPLICA-LEITURA-ESTRATEGIA-DE-DADOS.MD

Réplica de leitura e estratégia de dados

- **Status:** Aceito
- **Data:** 2026-11-10
- **Complementa:** ADR 0006 e ADR 0003

Contexto e problema

O consolidado lê um saldo que ele mesmo projeta. Com o cache quente, o Postgres quase não participa do GET. Com o cache frio, a leitura e a escrita da projeção dividem o mesmo primário. O blueprint deixa uma réplica parametrizável, desligada, e uma variável `DB_READER_URL` para o dia em que a leitura for o problema. Sem um gatilho, a variável vira "ligada em prod porque produção tem réplica".

Drivers

- O read model já é outro banco (o do consolidado). Réplica não é o primeiro passo de CQRS; CQRS já aconteceu.
- Estorno retroativo muda o passado. Estratégia que grava saldo acumulado em todas as linhas futuras paga esse estorno com contenção.
- Custo de uma réplica `db.t4g.medium`: USD 53/mês em `us-east-1`. Barato comparado ao RDS primário, caro comparado a um índice que faltava.
- Lag visível. Réplica atrasada mente o caixa.

Opções consideradas

1. Réplica ligada desde o dia 1

- Positivo: o caminho de código (`readOnly` → reader) nasce exercitado.
- Negativo: USD 53/mês para um `SUM` de 1.800 linhas que o primário faz em milissegundos. E um modo de falha novo (lag) no caminho do número que o comerciante lê.

2. Sem caminho de réplica no código, até precisar

- Positivo: menos um data source.
- Negativo: o dia em que o primário enche, a mudança é de código e de infra no mesmo incidente. A variável opcional custa poucas classes e zero dólar enquanto o URL está vazio.

3. Código pronto, réplica desligada, gatilho numérico

`DB_READER_URL` vazio: todo mundo no primário. Preenchido: `@Transactional(readOnly = true)` vai ao reader, com `LazyConnectionDataSourceProxy` para a conexão nascer na hora em que a transação sabe para qual lado vai. Escrita da projeção, outbox e lançamento ficam no primário sempre.

A réplica só é criada quando o documento 12 disparar: crédito de CPU da instância T caindo por causa de leitura, com hit ratio de cache ainda alto; ou p99 de cache miss acima de 300 ms com o tempo no Postgres; ou conexões de leitura empurrando o primário para 70% de `max_connections`.

Lag acima de 5 s: alarme, e a leitura volta ao primário.

Estratégia de dado em volta disso, na ordem em que se tenta:

1. Índice e uma sentença só para o acumulado (janela), agora.
2. Réplica de leitura do consolidado, no gatilho.
3. Snapshot mensal do acumulado, se a janela passar de 300 ms no p99 com história grande.
4. Aurora com reader autoscaling, se a quantidade de readers tiver de variar e o I/O justificar I/O-Optimized.
5. DynamoDB, OpenSearch ou Athena só quando a pergunta deixar de ser a chave `(comerciante, data)`. DynamoDB não é o destino natural do relatório com estorno retroativo.

- Positivo: o incidente de leitura tem um procedimento que não é “aumenta a instância”. O código não muda naquele dia; muda o tfvar e a variável.
- Negativo: código de routing pouco exercitado até o gatilho. O teste de integração com dois Postgres (Testcontainers) cobre o routing uma vez. Sem esse teste, a opção 3 é a opção 2 com uma classe morta.

Decisão

Opção 3. Default de reader: **zero**. RDS Proxy: **desligado**, pelo motivo do documento 12 (o wrapper já cobre segredo; na instância, também o failover; o Proxy cobre multiplexação, que ainda não falta).

Aurora não é o default. DynamoDB não substitui a projeção enquanto o produto tiver relatório de período e estorno em data antiga.

Limitação da AWS sobre read replica

A documentação do RDS com Secrets Manager diz que RDS for PostgreSQL com senha mestre gerenciada pelo Secrets Manager **não aceita read replica**. O módulo `rds-postgres` mantém `manage_master_user_password = true` e recusa `read_replica_count > 0` numa `validation` da variável e numa `precondition` da instância. `DB_READER_URL` não é injetada enquanto o output `reader_endpoint` for nulo.

Quando o gatilho da opção 3 disparar, a ordem compatível com a senha gerenciada é:

1. **RDS Multi-AZ DB Cluster** (recomendação). Um escritor e dois standbys legíveis, reader endpoint no `DB_READER_URL`. Não é a read replica proibida: é outro formato de implantação, e a senha continua no Secrets Manager. O custo deixa de ser os USD 53 de uma réplica e passa a ser o de três instâncias. O plugin do wrapper na instância/cluster segue `failover,awsSecretsManager`.
2. **Aurora PostgreSQL** com readers e autoscaling, se o número de readers tiver de variar no dia e o I/O justificar. Também convive com senha gerenciada.
3. **Credencial da réplica fora do Secrets Manager**, com rotação por Lambda. Aí a read replica clássica volta a existir, e o módulo teria de aceitar `manage_master_user_password = false`. É a saída que abandona a rotação nativa do RDS. Não é a recomendação.

Se `enable_rds_proxy` estiver ligado, o host de `DB_URL` contém `.proxy-`. Os serviços não usam o plugin `failover` contra esse host: o plugin abriria conexão na instância e furaria o proxy. Fica só `awsSecretsManager`. O proxy já promove o escritor.

Consequências

Positivas

- USD 53 ficam de fora da fatura até a evidência.
- O read model continua um só. Não se cria um terceiro banco “por simetria com CQRS”.

Negativas

- Alguém vai ligar a réplica porque “produção pede”. O parâmetro default 0 e este ADR são a resposta. Ligar sem o gatilho é mudança de decisão, não tuning.
- O teste com dois bancos precisa existir no consolidado. Se não existir, o agente de integração anota a dívida no lugar do teste, não aqui.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0031-API-GATEWAY-REST-REGIONAL.MD

API Gateway REST regional atrás do CloudFront

- **Status:** Aceito
- **Data:** 2026-11-10
- **Atualiza:** ADR 0022. Não substitui o CloudFront. O ADR 0018 continua substituído pelo 0022; este ADR é o passo seguinte da mesma borda.
- **Implementa:** seção 20 do blueprint

Contexto e problema

O ADR 0022 deixou o CloudFront como única porta pública e o ALB interno, sem IP público. O JWT era validado só na task. Isso cumpre o "o mesmo binário funciona no laboratório", e também deixa três controles para depois: rejeitar token inválido antes de gastar task, cota por consumidor de integração, e um contrato de borda que não muda quando o runtime atrás do ALB mudar.

O desafio pede critério de segurança para consumo de integração. Cliente final e PDV não são o mesmo consumidor. O primeiro autentica com o usuário do Cognito. O segundo precisa de teto próprio, identificável, sem abrir uma segunda API.

Drivers

- Token inválido não deve abrir conexão até o ECS. Authorizer na borda economiza vCPU e reduz a superfície que um token ruim enxerga.
- Throttle e quota por consumidor, não só rate-based por IP. NAT de shopping soma comerciantes no mesmo IP; o WAF não distingue o PDV do caixa.
- Validação do corpo antes da VPC. `tipo`, `valor` e data fora do contrato morrem com 400 no gateway.
- mTLS para parceiro B2B fica disponível no domínio customizado da REST API, sem virar requisito do comerciante no app.
- WAF regional no stage, porque o hostname `execute-api` continua alcançável. CloudFront não é a única URL que a AWS publica.
- Um catálogo. O stage `v1` é a versão. O export OpenAPI sai da própria API.
- Canary de stage e access log por rota, com `requestId` e o trace do X-Ray.
- Trocar ECS por EKS ou Lambda não muda o path público `/api/v1`.

O flag `enable_api_gateway = false` devolve o desenho do ADR 0022. O default dos três ambientes é `true`.

Opções consideradas

1. Sem gateway: CloudFront → VPC Origin → ALB

É o ADR 0022, e continua no código quando o flag está desligado.

- Positivo: um hop a menos. Sem cobrança por request. O laboratório e a AWS validam o JWT no mesmo lugar.
- Negativo: token inválido paga task, ALB e pool JDBC. Não há usage plan. Não há validação de JSON na borda. Não há canary de contrato. O único teto fino é o WAF por IP.

2. HTTP API (API Gateway v2)

- Positivo: cerca de USD 1,00 por milhão de requests em `us-east-1`, contra USD 3,50 da REST. JWT authorizer nativo, VPC Link v2 para o ALB, menos recurso para declarar. É a escolha certa quando não há usage plan, não há validação de body no gateway, não há canary e o volume passa de dezenas de milhões por mês — aí a diferença de USD 2,50 por milhão deixa de ser troco.
- Negativo, para este contrato: usage plan e API key são da REST API. Request validator com JSON Schema é da REST API. Canary de stage é da REST API. Cache de stage, se um dia for ligado com chave que inclua o token, é da REST API. mTLS de domínio customizado para parceiro B2B é da REST API. WAF a HTTP API passou a aceitar em parte do portfólio, mas o conjunto que o desafio pede (cota por consumidor + validação + canary + WAF no stage) não cabe na HTTP API sem completar o que falta na aplicação.

3. REST API regional

Região da API igual à do ALB (`us-east-1`). Não é edge-optimized: o anycast já é o CloudFront. Endpoint configuration `REGIONAL`.

- Positivo: usage plan, API key opcional por rota, request validator, modelo JSON Schema, canary, cache desligável, WAF no stage, mTLS no domínio customizado, access log com os campos da seção 20, authorizer Cognito com `authorization_scopes`.
- Negativo: USD 3,50 por milhão. Mais recurso no Terraform. O stage entra no path da integração privada (`/v1/api/v1/...`). O hostname do ALB interno não cabe em certificado público, então o hop até o ALB continua HTTP dentro da VPC, como já era no ADR 0022.

4. ALB com Lambda authorizer

- Positivo: um hop a menos que o gateway. O authorizer vê o JWT antes da task.
- Negativo: Lambda no caminho crítico, cold start, e nada de usage plan, validator ou canary. Outro runtime para operar. O ADR 0022 já recusou complicar o ALB.

5. Kong ou Apigee self-hosted

- Positivo: portal, transformação e política ricas.
- Negativo: cluster para dois paths estáveis. O custo fixo e a operação engolem a diferença de USD 12,50/mês entre REST e HTTP API em cinco milhões de requests. Entra na conversa se houver dezenas de APIs de produto, não neste caixa.

Decisão

Opção 3, na frente do que o ADR 0022 já construiu.

```
Cliente → CloudFront (+ WAF CLOUDFRONT)
       → API Gateway REST regional (+ WAF REGIONAL)
       → VPC Link v2 → ALB interno → ECS
```

Provider AWS 6.72 confirma o caminho. `aws_api_gateway_integration` tem `integration_target`. O `connection_id` é um `aws_apigatewayv2_vpc_link` (VPC Link v2: subnets e security group). O alvo é o ARN do ALB. Não há NLB. A documentação da AWS de novembro de 2025 descreve essa integração; o VPC Link v1 (`aws_api_gateway_vpc_link`) continua existindo e só aceita NLB. Não foi usado.

O que cada peça faz:

- **Authorizer Cognito** em todo método, com escopo `fluxo-caixa/lancamentos:write`, `fluxo-caixa/lancamentos:read` ou `fluxo-caixa/consolidado:read`. Token inválido ou escopo errado responde 401/403 no gateway. A task **continua** validando o mesmo JWT e lendo `comerciante_id`. Defesa em profundidade: o gateway não é o resource server, é a primeira peneira. O laboratório, sem gateway, não muda de código.
- **Throttle** de stage e de método. Em hml e prod o GET do consolidado fica acima de 50 req/s (80 e 100). Em dev fica em 20, de propósito.
- **Usage plan + API key** só para o mapa `api_consumers`. Vazio por default. `api_key_required` é por rota e nasce `false`: o comerciante não carrega API key.
- **Validator** de corpo e parâmetros no POST de lançamento. O POST de estorno valida só parâmetros: o corpo é opcional, como a seção 7. O model `EstornarLancamento` documenta `motivo` (até 255, opcional), e o serviço aplica esse limite quando o JSON vem. Ligar a validação de corpo nesse método rejeitaria a ausência.
- **WAF regional** com Common, Known Bad Inputs, SQLi e IP Reputation, mais a regra que bloqueia o request cujo header `X-Origin-Verify` não é o valor `current` nem o `previous`. O CloudFront grava o valor `current` num custom header da origem `api-gateway` e usa `AllViewerExceptHostHeader`: o `Host` tem de ser o do `execute-api`, senão a REST API responde 403; o header customizado da origem substitui qualquer valor que o cliente tenha mandado com o mesmo nome.
- **Rotação** do segredo no Secrets Manager, CMK própria, Lambda no protocolo `createSecret` / `setSecret` / `testSecret` / `finishSecret`. A Lambda gera o valor novo, grava `previous` com o current antigo, atualiza o custom header do CloudFront (distribuição achada pela tag `Name`) e os dois byte matches da regra. O valor anterior continua válido até a rotação seguinte, que é a janela em que a distribuição ainda propaga. `rotate_immediately = false` para o apply não girar o header. Dev 90 dias, hml e prod 30, no mesmo ritmo da senha do RDS.
- **Stage** `v1`: access log JSON, X-Ray, métrica detalhada, cache desligado, canary parametrizável. O canary aponta para o mesmo deployment enquanto não houver um segundo deployment; `percent_traffic` 0 deixa o bloco pronto sem desviar tráfego.
- **Path do stage.** A integração privada inclui o nome do stage no path que o ALB vê (`/v1/api/v1/...`). O listener reescreve `^/v1(/.*)$` para `$1` só quando `enable_api_gateway` está ligado. Sem o prefixo, a regra não casa e o path do ADR 0022 segue inteiro. O timeout de integração é 10 s. A URI leva o path e os path params; os headers `Authorization`, `traceparent`, `tracestate`, `X-Correlation-Id` e

`Idempotency-Key` são mapeados. `X-Origin-Verify` é limpo antes do ALB para não cair no log da aplicação.

mTLS de parceiro não é criado nesta entrega. Não há domínio de parceiro nem truststore. A REST API é o lugar onde isso entra, com domínio customizado e S3 truststore, sem redesenhar o ECS.

Consequências

- Hop a mais. A ordem de grandeza publicada pela AWS para o gateway regional fica em 10–30 ms. O SLO de latência deste sistema continua medido no serviço (p95 do POST < 200 ms, p99 < 500 ms; GET de consolidado p95 < 100 ms com cache). O comerciante percebe o gateway em cima do RTT já discutido no ADR 0021. Não se aperta o SLO do processo para "absorver" o gateway: os dois números são séries diferentes. O alarme de `Latency` p99 do gateway está em 1 s; o de `IntegrationLatency` p99, em 800 ms.
- Custo variável. Com a premissa já usada no CloudFront (0,2 / 1 / 5 milhões de requests por mês), a REST API custa USD 0,70 / 3,50 / 17,50. A HTTP API, na mesma premissa, custaria USD 0,20 / 1,00 / 5,00. A diferença em produção é USD 12,50/mês. O WAF regional acrescenta cerca de USD 10 de piso mais USD 0,60 por milhão. A conta fechada está no documento de custos.
- O bootstrap do header é `secret_string_wo` com senha efêmera. O resource da versão não guarda o valor. CloudFront e o WAF leem `AWSCURRENT` via data source, então o apply depois da rotação acompanha o par vigente em vez de reverter ao sorteio inicial. O header continua no state da distribuição, o par no state do data source e nas strings da regra. `secret_string_wo_version` permanece 1; incrementá-lo regrava o segredo e deve ser uma decisão explícita, não um efeito colateral do plan.
- `ListDistributions` da Lambda é `Resource: "*"`. A API da AWS não aceita ARN nesse action. O update da distribuição está limitado a `distribution/*` da conta, e o código recusa qualquer distribuição cuja tag `Name` não seja a do ambiente.
- Cliente browser em outra origem continua sem CORS, como já era no ALB. Não foi pedido.
- OpenAPI do gateway e OpenAPI do springdoc podem divergir se alguém alterar o DTO e esquecer o model. O model é a peneira da borda; o Bean Validation continua sendo a regra do domínio.

Custos que esta decisão assume

Preço de lista em `us-east-1`, o mesmo já usado no documento de custos para WAF (USD 5 por Web ACL, USD 1 por regra, USD 0,60 por milhão) e o preço que a seção 20 fixa para a API (REST USD 3,50/milhão, HTTP USD 1,00/milhão). VPC Link v2 não entra com linha horária: o custo que se evita é o NLB que a integração antiga exigia. Lambda de rotação: uma invocação por período, abaixo de um centavo. Segredo extra: USD 0,40/mês. CMK extra: USD 1/mês.